MEDSECURE: A BLOCKCHAIN AND ARTIFICIAL INTELLIGENCE-POWERED

FRAMEWORK FOR PHARMACEUTICAL SUPPLY CHAIN INTEGRITY

BY

Tanisha Katariya (22C21028)

Ayushi Patel (22C21067)

Krutang Panchal (22C21045)

Vraj Sondagar (22C21117)

Guided By

Prof. Shivangi Matieda

A REPORT

SUBMITTED TO

ITM SLS Baroda University

in partial fulfilment of the requirements

for the degree of

BACHELOR OF TECHNOLOGY

Computer Science & Engineering

School of Computer Science Engineering & Technology

April 2026

DECLARATION OF ORIGINALITY

We declare that this report entitled "MEDSECURE: A BLOCKCHAIN AND ARTIFICIAL INTELLIGENCE-POWERED FRAMEWORK FOR PHARMACEUTICAL SUPPLY CHAIN INTEGRITY" is our own work except as cited in the references. This report has not been accepted for any degree and is not being submitted concurrently in candidature for any degree or award.

Signature : _____

Names : Tanisha Katariya, Ayushi Patel, Krutang Panchal, Vraj Sondagar

Date : April 2026

Signature of Supervisor: _____

Date: _____

Place: _____

ACKNOWLEDGEMENTS

The successful completion of MedSecure has been made possible through the guidance, support, and encouragement of several individuals, to whom the team would like to express their deepest gratitude.

First and foremost, the team would like to express sincere thanks to the project supervisor, Prof. Shivangi Matieda, for the invaluable guidance, constructive feedback, and unwavering encouragement throughout the duration of this project. The supervisor's expertise and insights were instrumental in shaping the direction and quality of this work.

The team is also grateful to the Dean, School Of Computer Science And Technology, Prof. (Dr.) Pradeep Laxkar, the Head of Department, Prof. Gaurav Kulkarni and all the faculty members of the Department of Computer Science Engineering, ITM SLS Baroda University, Vadodara, Gujarat, for their consistent academic support and for providing an environment that encouraged innovation and research.

The team would further like to acknowledge the broader academic community whose prior research on blockchain-based pharmaceutical traceability, computer vision authentication, and decentralised systems provided the foundation upon which MedSecure was designed and developed.

Finally, the team would like to thank their families and friends for their patience, motivation, and moral support throughout this demanding yet rewarding academic journey.

ABSTRACT

The global pharmaceutical supply chain is confronted with an escalating crisis of counterfeit and substandard medicines, which the World Health Organization estimates account for approximately 10 per cent of medicines in low- and middle-income countries, resulting in hundreds of thousands of preventable deaths each year. Existing traceability solutions remain fragmented, relying upon centralised databases that are susceptible to tampering, single points of failure, and limited cross-stakeholder transparency. Blockchain alone does not address the problem of physical packaging authentication, as a counterfeit product may carry a legitimate Batch ID copied from authentic packaging.

This report presents MedSecure, a decentralised framework that integrates Ethereum blockchain technology with an artificial intelligence-driven computer vision module to deliver end-to-end pharmaceutical supply chain integrity and real-time counterfeit detection. Manufacturers register medicine batches immutably on-chain with IPFS-linked documentation, and a Solidity smart-contract-enforced state machine governs ownership transfers across manufacturers, distributors, and pharmacies. At the point of sale, consumers and pharmacists enter Batch IDs to trigger real-time on-chain verification, supplemented by an OpenCV-based packaging authentication pipeline employing ORB feature matching and the Structural Similarity Index Measure (SSIM).

MedSecure addresses the principal limitations of existing systems — lack of end-to-end transparency, absence of consumer-facing verification, and vulnerability to Batch ID cloning and physical packaging counterfeiting — through a defence-in-depth architecture combining immutable on-chain state management, tamper-evident IPFS storage, keyed Batch ID integrity verification, and AI-powered packaging authentication. Functional testing, security analysis, and comparative evaluation confirmed that MedSecure provides a scalable, cost-effective, and regulator-compliant solution, and is the only evaluated system providing public blockchain deployment, direct consumer access, computer vision authentication, and Batch ID locking simultaneously.

TABLE OF CONTENTS

TITLE		**i**
DECLARATION OF ORIGINALITY		**ii**
ACKNOWLEDGEMENTS		**iii**
ABSTRACT		**iv**
TABLE OF CONTENTS		**v**
LIST OF FIGURES		**vii**
LIST OF TABLES		**ix**
LIST OF ABBREVIATIONS		**x**
CHAPTER 1 INTRODUCTION		**1**
1-1	Problem Statement and Motivation	1
1-2	Objectives	3
1-3	Project Scope and Direction	4
1-4	Highlights Of Achievements	5
1-5	Report Organisation	5
CHAPTER 2 LITERATURE REVIEW		**6**
2-1	Methodology	6
2-2	Review of Prior Work	6
2-3	Summary	10
CHAPTER 3 SYSTEM REQUIREMENTS		**11**
3-1	Introduction	11

3-2	Functional Requirements	11
3-3	Non-Functional Requirements	15
3-4	Software and Hardware Requirement	16
3-5	Summary	17
CHAPTER 4 SYSTEM DESIGN		**18**
4-1	Introduction	18
4-2	Proposed System	18
4-3	Use Case Diagram	20
4-4	Class Diagram	21
4-5	Sequence Diagram	22
4-6	Activity Diagram	23
4-7	Data Flow Diagram	24
4-8	Entity-Relationship Diagram	25
4-9	Summary	26
CHAPTER 5 IMPLEMENTATION		**27**
5-1	Introduction	27
5-2	Smart Contract and Blockchain Layer	27
5-3	IPFS Decentralised Storage Layer	28
5-4	Application Service Layer	28
5-5	Frontend Presentation Layer	29
5-6	Core Algorithms	29

5-7	Architectural Components	34
5-8	Feature Extraction	35
5-9	Packages and Libraries Used	36
CHAPTER 6 SYSTEM TESTING		**37**
6-1	Introduction	37
6-2	Test Cases	37
6-3	Results	40
6-4	Performance Evaluation	41
6-5	Summary	42
CHAPTER 7 USER INTERFACE SCREENSHOTS		**43**
7-1	Introduction	43
7-2	Home Page and Landing Interface	43
7-3	Manufacturer Portal	44
7-4	Distributor Portal	46
7-5	Pharmacy Portal	46
7-6	Consumer Verification Portal	47
7-7	Summary	49
CONCLUSION		50
REFERENCES		52
APPENDICES		

LIST OF FIGURES

Figure Number	Title	Page
Figure 4-1-F1	MedSecure System Architecture Overview	19
Figure 4-2-F1	Use Case Diagram — Actor Interactions	20
Figure 4-3-F1	Class Diagram — Smart Contract and Service Layer	21
Figure 4-4-F1	Sequence Diagram — Batch Registration and Verification Flow	22
Figure 4-5-F1	Activity Diagram — Consumer Verification Pipeline	23
Figure 4-6-F1	Data Flow Diagram — Level 0 Context Diagram	24
Figure 4-7-F1	Data Flow Diagram — Level 1 Decomposition	24
Figure 4-8-F1	Entity-Relationship Diagram — Off-Chain PostgreSQL Schema	25

LIST OF TABLES

Table Number	Title	Page
Table 2-1-T1	Literature Review — Summary of Prior Work and Gaps	6
Table 3-1-T1	Role-Based Access Control Permissions	14
Table 3-2-T1	Software Requirements	16
Table 3-3-T1	Hardware Requirements	17
Table 6-1-T1	Test Case — Batch Registration	37
Table 6-2-T1	Test Case — Ownership Transfer	38
Table 6-3-T1	Test Case — Batch ID Verification and Locking	39
Table 6-4-T1	Test Case — CV Authentication	39
Table 6-5-T1	Performance Characteristics	41
Table 6-6-T1	Comparative Evaluation of MedSecure vs Existing Systems	41

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CID	Content Identifier
CLAHE	Contrast Limited Adaptive Histogram Equalisation
CV	Computer Vision
DSCSA	Drug Supply Chain Security Act
IPFS	InterPlanetary File System
JWT	JSON Web Token
NFT	Non-Fungible Token
ORB	Oriented FAST and Rotated BRIEF
RPC	Remote Procedure Call
SHA	Secure Hash Algorithm
SSIM	Structural Similarity Index Measure
UI	User Interface
URL	Uniform Resource Locator
WHO	World Health Organization

CHAPTER 1 : INTRODUCTION

1-1 : Problem Statement and Motivation

Counterfeit and substandard medicines represent one of the most severe and persistent threats to global public health. The World Health Organization reported in 2017 that falsified medical products are prevalent in both developed and developing economies, with an estimated market value exceeding USD 200 billion annually (World Health Organization 2017). Patients who consume counterfeit medicines receive either no therapeutic benefit or, in many cases, actively harmful substances, contributing to antimicrobial resistance, treatment failure, and death. In low- and middle-income countries, falsified medicines are estimated to cause over 250,000 deaths each year, with a disproportionate impact on children suffering from malaria and pneumonia.

The pharmaceutical supply chain is inherently complex, spanning manufacturers, distributors, wholesalers, pharmacies, and consumers across national and international borders. This complexity creates multiple points of vulnerability at which counterfeit products may be introduced. Traditional regulatory approaches rely on paper-based certificates of analysis and centralised digital databases, both of which are susceptible to forgery and tampering. The absence of interoperability between stakeholder systems means that end-to-end traceability from manufacturer to consumer remains largely unachievable in practice under existing approaches.

Blockchain technology offers a promising architectural foundation for supply chain transparency. Its core properties of immutability, decentralisation, and cryptographic verifiability make it well-suited to maintaining tamper-proof records of pharmaceutical product provenance. Several pilot projects and industry initiatives, including the MediLedger consortium blockchain and the IBM-FDA Drug Supply Chain Security Act (DSCSA) pilot, have demonstrated the technical feasibility of blockchain-based drug traceability within regulated supply chains (MediLedger Project 2020). However, three critical limitations persist across existing solutions. First, most deployed systems are enterprise-focused and permissioned, excluding direct consumer-level access to product verification. Second, blockchain alone does not address physical packaging authentication — a counterfeit product may carry a valid Batch ID copied from legitimately sold packaging. Third, the majority of proposed systems remain conceptual or narrowly scoped, lacking the integration of decentralised document

storage, consumer verification, and AI-based physical product authentication in a single deployable system.

Artificial intelligence, particularly computer vision, has demonstrated significant promise in detecting packaging anomalies indicative of counterfeiting (Li et al. 2020). Feature-matching algorithms such as ORB and perceptual similarity metrics such as SSIM have been applied to product authentication tasks with encouraging results. When combined with blockchain-based provenance verification, computer vision provides an independent layer of authenticity assurance that is substantially more difficult for counterfeiters to circumvent simultaneously.

The convergence of these challenges — the inadequacy of existing systems, the limitations of blockchain-only approaches, and the untapped potential of computer vision for physical packaging verification — motivates the development of MedSecure as a unified, end-to-end solution.

1-2 : Objectives

The primary objective of MedSecure was to design, implement, and evaluate a decentralised pharmaceutical supply chain integrity and counterfeit medicine detection framework that addresses the key limitations of existing solutions. The specific objectives were as follows:

1. To develop and deploy Ethereum Solidity smart contracts on the Sepolia public testnet that enforce an immutable, role-based state machine for medicine batch lifecycle management, encompassing registration, ownership transfer, and sale completion.
2. To integrate IPFS decentralised storage for tamper-evident off-chain storage of batch documentation, compliance certificates, and reference packaging images, with on-chain CID anchoring to guarantee content integrity.
3. To implement a computer vision authentication module using OpenCV that employs ORB feature matching and SSIM to compare live consumer-captured packaging images against manufacturer-uploaded reference images, and returns a composite confidence score and classification.

4. To construct a consumer-facing verification portal accessible without a Web3 wallet that executes the complete on-chain verification and computer vision authentication pipeline and presents results in plain language.
5. To implement a dual-layer audit trail logging system that records all critical supply chain events both on-chain for regulatory immutability and off-chain in PostgreSQL for fast query access and compliance reporting.
6. To conduct a security analysis, functional testing, and comparative evaluation of MedSecure against representative existing pharmaceutical traceability and anti-counterfeiting systems.

1-3 : Project Scope and Proposed Approach

MedSecure was scoped as a software application development project comprising four tightly integrated layers: a blockchain and smart contract layer, a decentralised storage layer, an application service layer, and a frontend presentation layer. The system targeted four primary actors: Manufacturer, Distributor, Pharmacy, and Consumer.

The Blockchain and Smart Contract Layer deployed Solidity smart contracts on the Ethereum Sepolia testnet, governing batch registration and the complete ownership transfer lifecycle through an enforced linear state machine (MANUFACTURED → SENT_TO_DISTRIBUTOR → READY_FOR_SALE → SOLD). Role-based access control ensured that each actor could only invoke functions appropriate to their assigned role.

The Decentralised Storage Layer utilised IPFS to store all batch documentation and reference packaging images. Each uploaded file received a cryptographic content identifier (CID) derived from the hash of its contents, which was subsequently anchored on-chain during batch registration. Any post-registration modification to a stored file produces a different CID, rendering tampering immediately detectable.

The Application Service Layer was implemented as a stateless Next.js API service interfacing with the Ethereum network via the Viem and Wagmi libraries. A PostgreSQL database served as a read-optimised cache of on-chain data to reduce query

costs and improve response latency. The computer vision module was implemented as a separate Python microservice using OpenCV.

The Frontend Presentation Layer provided four role-specific portals for the Manufacturer, Distributor, Pharmacy, and Consumer actors. The Consumer Portal was explicitly designed to require no blockchain wallet, making verification accessible to any member of the public with a smartphone or browser.

1-4 : Highlights of Achievements

The principal achievements of the MedSecure project were as follows. All six core functional modules — batch registration, ownership transfer, Batch ID lifecycle management, computer vision authentication, consumer verification, and audit trail logging — were successfully implemented and tested. Smart contract unit tests implemented in Foundry achieved full coverage of all state transition paths, including expected revert conditions. The composite ORB and SSIM computer vision scoring demonstrated reliable discrimination between authentic and counterfeit packaging samples. The system achieved target performance characteristics of under 2 seconds for Batch ID verification-to-on-chain data retrieval and under 5 seconds for the complete CV authentication pipeline. Comparative evaluation confirmed that MedSecure is the only evaluated system providing public blockchain deployment, direct consumer access, computer vision authentication, and Batch ID locking simultaneously.

1-5 : Report Organisation

The remainder of this report is organised as follows. Chapter 2 surveys related prior work in blockchain-based pharmaceutical traceability and AI-based counterfeit detection, identifying the key gaps that MedSecure addresses. Chapter 3 documents the functional and non-functional requirements of the system, together with the software and hardware requirements. Chapter 4 presents the complete system design, encompassing the system architecture and all UML and data flow diagrams. Chapter 5 describes the implementation of each system layer and all core algorithms in detail. Chapter 6 reports the system testing results, performance evaluation, and comparative evaluation. The report concludes with a summary of achievements, limitations, and directions for future work.

CHAPTER 2 : LITERATURE REVIEW

2-1 : Methodology

The literature review was conducted using a structured search methodology targeting peer-reviewed conference papers, journal articles, and verified industry pilot reports relevant to blockchain-based pharmaceutical supply chain management, AI-based counterfeit detection, and hybrid blockchain-AI authentication systems. Searches were performed across the IEEE Xplore Digital Library, ACM Digital Library, Google Scholar, and PubMed databases using keyword combinations including 'blockchain pharmaceutical supply chain,' 'drug traceability blockchain,' 'counterfeit medicine detection AI,' 'computer vision packaging authentication,' and 'Ethereum smart contract healthcare.'

Inclusion criteria required that works directly addressed pharmaceutical counterfeiting, drug traceability, or supply chain authentication using blockchain, AI, or a combination thereof. Works were evaluated for methodology, key findings, identified limitations, and the degree to which their approach overlapped with or fell short of the MedSecure framework. A total of twelve verified prior works and industry projects were identified and reviewed. The findings are synthesised in the comparative table below, followed by a narrative discussion of each work.

2-2 : Review of Prior Work

Table 2-1-T1 presents a structured comparison of twelve prior works, documenting the methodology, key findings, and identified gaps for each, together with an explanation of how MedSecure addresses those gaps.

Table 2-1-T1: Literature Review — Summary of Prior Work and Research Gaps

Author(s), Year	Title	Methodology & Key Findings	Gap Identified	How MedSecure Fills the Gap
Kumar & Tripathi (2019)	Traceability of Counterfeit	Ethereum blockchain with encrypted QR; complete drug	No decentralised document	MedSecure adds IPFS for tamper-proof

	Medicine Supply Chain	traceability demonstrated.	storage; no consumer verification mechanism.	storage and Batch ID-based consumer verification.
Tseng et al. (2018)	Governance on Drug Supply Chain via Gcoin	Conceptual Gcoin public ledger framework; improved governance and accountability.	Purely theoretical; no deployable implementation.	MedSecure is a fully deployable Web3 system on public Ethereum Sepolia.
MediLedger Project (2020)	Blockchain for Prescription Drug Verification	Consortium permissioned blockchain for DSCSA compliance; regulatory verification demonstrated.	Enterprise-focused; no consumer-level access.	MedSecure enables direct consumer drug verification via Batch ID verification.
IBM-FDA Pilot (2017)	DSCSA Pilot via Hyperledger	Closed Hyperledger pilot; improved traceability speed and data integrity demonstrated.	Closed system; not scalable or open to consumers.	MedSecure uses public Ethereum testnet for open, transparent verification.
Rai / BBTCD (2022)	Blockchain-Based Traceability of	Ethereum + IPFS for batch tracking and document storage; improved traceability shown.	No AI-based authentication layer for product verification.	MedSecure adds dual authentication via blockchain and CV-AI

	Counterfeited Drugs			packaging analysis.
Abbas et al. (2020)	Blockchain + ML Drug Supply Chain	Hyperledger Fabric with LightGBM and N-gram models; ML fraud detection demonstrated.	ML not applied to consumer-facing physical packaging verification.	MedSecure deploys CV-AI for real-time packaging verification by consumers.
Musamih et al. (2021)	Blockchain for Drug Traceability in Healthcare	Ethereum + IPFS for drug provenance; data integrity and trust guaranteed.	No Batch ID-based consumer verification or CV-AI packaging authentication.	MedSecure adds Batch ID consumer access and CV-AI scan to complete verification.
Zhu et al. (2020)	Blockchain Solution for Medication Anti-Counterfeiting	Smart contract access control for medication tracking; transparent immutable records.	No IPFS integration; no AI-based verification layer.	MedSecure integrates IPFS for documents and CV-AI for packaging authentication.
Singh et al. (2020)	IoT Blockchain for Temperature Monitoring	IoT sensor blockchain framework; real-time monitoring and authenticity tracking.	Heavy IoT hardware dependency; weak on-chain enforcement logic.	MedSecure enforces trust via immutable on-chain smart contracts without IoT hardware.

Sylim et al. (2018)	Blockchain for Detecting Falsified Drugs	Pharmacosurveillance DApp; consumer-facing scan of drug distribution history.	Simulated network only; no smart contract-enforced logic or IPFS storage.	MedSecure deploys working smart contracts on Sepolia with IPFS storage.
Uddin (2021)	Hyperledger Fabric Drug Traceability System	Permissioned Medledger with chaincode linked to IPFS/Swarm; high integrity.	Private blockchain; no consumer verification or CV-AI packaging analysis.	MedSecure uses public Ethereum, adds consumer Batch ID verification and CV-AI.
Gomasta et al. (2023)	PharmaChain: Blockchain Drug Supply Chain Provenance	Hyperledger with PBFT consensus, ECDSA double signing; tamper-proof provenance.	Permissioned consortium; no CV-AI or QR-based visual authentication.	MedSecure extends with public blockchain and CV-AI physical packaging verification.

A review of the existing literature reveals a consistent and clear pattern across all twelve works. Whilst blockchain demonstrably improves supply chain transparency, traceability, and tamper resistance, existing solutions remain constrained by one or more of the following limitations: enterprise-focused or permissioned architectures that exclude consumer access; purely conceptual frameworks without deployable implementations; the absence of decentralised document storage; no integration of computer vision or AI for physical packaging verification; and reliance on closed or simulated environments that do not reflect real public blockchain deployment conditions.

The works of Kumar and Tripathi (2019) and Rai (2022) are the most technically similar to MedSecure in employing both Ethereum and, in Rai's case, IPFS. However, neither incorporated any form of AI-based physical packaging verification, nor did they provide the consumer-facing verification interface that is central to MedSecure's approach. The machine learning-integrated work of Abbas et al. (2020) demonstrates the value of AI in pharmaceutical fraud detection but applies ML exclusively to supply chain transaction anomaly detection rather than to the physical verification of packaging at the consumer's point of purchase.

The PharmaChain system of Gomasta et al. (2023) is the most architecturally sophisticated of the reviewed systems, employing PBFT consensus and ECDSA double signing for strong cryptographic provenance. However, its permissioned consortium architecture fundamentally limits its openness, and it provides no mechanism for consumers to independently verify the physical packaging of a product they are purchasing.

2-3 : Summary

The literature review confirms that MedSecure occupies a novel position in the existing body of work by simultaneously addressing the full set of recurring gaps: open public blockchain deployment on Ethereum Sepolia, Solidity-enforced smart contract state management, IPFS-backed tamper-evident document storage, Batch ID lifecycle management with cryptographic integrity verification, consumer-facing verification without a Web3 wallet requirement, and AI-powered computer vision packaging authentication using ORB and SSIM. No prior reviewed work provided all these capabilities within a single integrated, deployable system.

CHAPTER 3 : SYSTEM REQUIREMENTS

3-1 : Introduction

This chapter documents the complete requirements specification for MedSecure, encompassing both functional and non-functional requirements derived from analysis of the pharmaceutical supply chain workflow and the specific objectives of the project. Requirements were identified through a combination of stakeholder role analysis, examination of pharmaceutical regulatory compliance standards (including the DSCSA), and gap analysis of existing solutions identified in the literature review.

3-2 : Functional Requirements

Functional requirements define the specific behaviours, operations, and capabilities that the MedSecure system provided to its four primary actors: Manufacturer, Distributor, Pharmacy, and Consumer.

3-2-1 : User Registration and Wallet-Based Authentication

MedSecure employed Ethereum wallet addresses as the primary identity mechanism for all supply chain stakeholders. Manufacturers, Distributors, and Pharmacies were required to connect a compatible Web3 wallet (such as MetaMask) to access their respective portals. The system validated wallet signatures on every session to ensure non-repudiability. Consumers were exempted from wallet requirements and interacted with the system through a Batch ID verification interface.

- Wallet signature verification was performed on every login attempt.
- Role assignment (MANUFACTURER, DISTRIBUTOR, PHARMACY, CONSUMER) was stored and enforced by the Solidity smart contract.
- Session management was implemented using JSON Web Tokens (JWT) bound to the authenticated wallet address.
- Re-authentication was triggered upon detection of a wallet address change.

3-2-2 : Medicine Batch Registration

Manufacturers registered new medicine batches through the system portal. Each batch was assigned a unique on-chain identifier and linked to its complete lifecycle

metadata. The registration process involved uploading supporting documents and packaging images to IPFS, writing metadata to the Ethereum smart contract, and generating a Batch ID encoding the batch identifier.

- Input fields included: batch name, manufacturing date, expiry date, composition, unit count, packaging images, and compliance certificates.
- Documents were uploaded to IPFS and the resulting content identifier (CID) was stored on-chain as the canonical reference.
- The smart contract emitted a BatchRegistered event containing the batchId, IPFS hash, manufacturer wallet address, and timestamp.

3-2-3 : Ownership Transfer and Supply Chain Tracking

The smart contract enforced strict ownership rules throughout the supply chain. Only the current on-chain owner of a batch was permitted to initiate a transfer to the next party. The system recorded the complete chain of custody from Manufacturer to Distributor to Pharmacy, with each transfer immutably logged on the Ethereum blockchain.

- Batch status followed a strict linear transition: MANUFACTURED → SENT_TO_DISTRIBUTOR → READY_FOR_SALE → SOLD.
- Each transfer emitted an OwnershipTransferred event on-chain.
- Unauthorised transfer attempts were rejected by the smart contract with a descriptive revert message.

3-2-4 : Batch ID Generation, Verification, and Locking

Every registered batch received a unique BATCH ID. Pharmacies entered the Batch ID to retrieve batch information and verify ownership prior to marking a unit as sold. Once a unit was sold, its Batch ID was permanently locked to prevent reuse or counterfeiting through Batch ID cloning.

- Batch IDs were permanently locked upon transition to SOLD status; subsequent scans returned an ALREADY_SOLD warning response.
- Expired or tampered Batch IDs were flagged with an appropriate error response.

3-2-5 : Computer Vision Packaging Authentication

A Python-based computer vision module powered by OpenCV compared live packaging images captured by the consumer or pharmacy against manufacturer-uploaded reference images stored on IPFS. The system computed a composite confidence score; batches scoring above the configured threshold were classified as authentic, whilst those falling below triggered a counterfeit alert.

- Reference images were retrieved from IPFS using the on-chain IPFS hash stored during batch registration.
- Pre-processing steps included greyscale conversion, Gaussian blur, CLAHE enhancement, and Canny edge detection.
- Feature comparison was performed using ORB descriptor matching and SSIM.
- A composite confidence score was computed; the default authenticity threshold was set at 75%.
- The result (AUTHENTIC, SUSPICIOUS, or COUNTERFEIT) was logged to PostgreSQL and appended to the on-chain audit trail.

3-2-6 : Consumer Verification Portal

Consumers were able to verify medicine authenticity without requiring a blockchain wallet. The consumer portal provided a simplified Batch ID verification interface accessible via any modern browser or mobile device.

- No wallet connection was required for the consumer verification workflow.
- Batch ID verification triggered automatic on-chain batch data retrieval.
- Results displayed to the consumer included: batch name, manufacturer, expiry date, and authenticity status.

3-2-7 : Audit Trail and Event Logging

All significant actions within MedSecure were recorded in a dual-layer audit trail. Critical events were stored immutably on the Ethereum blockchain as smart contract events, whilst detailed operational logs were persisted in PostgreSQL for fast querying and compliance reporting.

- On-chain events included: BatchRegistered, OwnershipTransferred, BatchSold, and CVVerificationLogged.
- Audit logs were designated as read-only; deletion was not permitted within the system.
- The full lifecycle history of any batch was queryable by batchId for regulatory or investigative purposes.

3-2-8 : Role-Based Access Control

Access to system features was strictly governed by the user's on-chain role. The smart contract enforced role checks on every state-changing operation.

Table 3-1-T1: Role-Based Access Control Permissions for Each Actor in MedSecure

Role	Permitted Actions
Manufacturer	Register batch, upload documents to IPFS, generate Batch ID, initiate transfer to Distributor.
Distributor	Accept batch transfer, initiate transfer to Pharmacy, view current supply chain status.
Pharmacy	Accept batch from Distributor, mark batch as READY_FOR_SALE, enter Batch ID, mark batch as SOLD.
Consumer	Verify Batch ID, view batch information and CV verification result (read-only, no wallet required).

3-3 : Non-Functional Requirements

3-3-1 : Security

Given the critical nature of pharmaceutical supply chain data, security was the highest-priority non-functional concern in MedSecure. All blockchain transactions required a valid wallet signature; private keys never left the client device. Smart

contracts were designed against known Solidity vulnerabilities including reentrancy, integer overflow, and access control flaws. IPFS content was addressed by cryptographic hash, making tampering immediately detectable. API endpoints were protected by JWT authentication with wallet-address binding. Batch ID locking prevented double-spend and replay attacks on sold units. Input validation and sanitisation were applied on all API endpoints to prevent injection attacks.

3-3-2 : Performance

MedSecure was designed to handle real-time verification requests from consumers and pharmacies without noticeable latency under standard operating conditions. Batch ID verify-to-on-chain data retrieval targeted a response time of less than 2 seconds. The CV authentication pipeline targeted a processing time of less than 5 seconds per verification request. Smart contract read operations targeted a response of less than 500 milliseconds via cached RPC calls. PostgreSQL queries on indexed columns targeted a response of less than 100 milliseconds. The Next.js frontend targeted an initial page load of less than 3 seconds on a standard 4G mobile connection.

3-3-3 : Scalability

The system architecture was designed to scale horizontally to accommodate increasing transaction volumes. Stateless Next.js API services could be replicated behind a load balancer without architectural changes. PostgreSQL read replicas were supported for increased query loads. IPFS handled distributed file storage natively, eliminating a centralised storage bottleneck. Smart contract functions were gas-optimised for cost-effective on-chain operation at scale.

3-3-4 : Reliability and Availability

The blockchain layer provided inherent high availability through its decentralised node architecture. IPFS ensured content availability through distributed content pinning. PostgreSQL was configured with automated daily backups and point-in-time recovery capability. Target system uptime was set at 99.5%, excluding scheduled maintenance windows.

3-3-5 : Compliance and Auditability

Immutable on-chain records were designed to satisfy regulatory traceability requirements for pharmaceutical products. Audit logs were retained for a minimum of

seven years in accordance with pharmaceutical compliance standards. Personally identifiable information (PII) was limited to wallet addresses; no names or contact details were stored on-chain.

3-4 : Software and Hardware Requirements

Table 3-2-T1: Software Requirements

Tool / Technology	Purpose
Solidity (v0.8.x)	Smart contract development for Ethereum blockchain
Foundry	Smart contract compilation, testing, and deployment
Next.js (v14)	Application service layer — RESTful API and frontend
Viem and Wagmi	Ethereum wallet connection, transaction signing, contract interaction
PostgreSQL (v15)	Off-chain read-optimised cache and audit log storage
Python (v3.11) + OpenCV	Computer vision authentication microservice
IPFS / Pinata	Decentralised storage for batch documentation and packaging images
Node.js (v20)	Server-side JavaScript runtime
MetaMask	Browser-based Web3 wallet for stakeholder authentication
Hardhat / Remix IDE	Supplementary smart contract development and testing
Git and GitHub	Version control and collaborative development

Table 3-3-T1: Hardware Requirements

Hardware Component	Specification / Purpose

Development Machines	Personal computers with minimum 8 GB RAM, 4-core CPU, and stable internet connection.
Smartphone (Consumer testing)	Any modern Android or iOS device with a camera and browser for Batch ID verification CV verification testing.
Stable Internet Connection	Required for Ethereum testnet RPC calls, IPFS pinning, and collaborative development via GitHub.

3-5 : Summary

This chapter documented the complete requirements specification for MedSecure, encompassing eight functional requirement areas governing all four actor roles, five non-functional requirement categories, and the complete software and hardware environment. The requirements reflect the dual objectives of MedSecure: to provide a robust, tamper-resistant supply chain record that satisfies pharmaceutical regulatory standards, and to deliver a consumer-accessible, real-time verification experience that does not require specialist knowledge or equipment.

CHAPTER 4 : SYSTEM DESIGN

4-1 : Introduction

This chapter presents the complete system design of MedSecure, encompassing the overall system architecture and all UML diagrams including the use case diagram, class diagram, sequence diagram, activity diagram, data flow diagrams at both the context level and decomposed level, and the entity-relationship diagram for the off-chain PostgreSQL schema. The system design was developed following a top-down approach, commencing with the high-level architectural decomposition before descending to the interaction and data-level specifications. All diagrams were designed such that a reader could reconstruct the core system components and their interactions from this chapter alone.

4-2 : Proposed System Architecture

MedSecure was designed as a layered, modular architecture comprising four principal layers: the Blockchain and Smart Contract Layer, the Decentralised Storage Layer, the Application Service Layer, and the Frontend Presentation Layer. Each layer encapsulates a distinct set of responsibilities and communicates with adjacent layers through well-defined interfaces, enabling independent scalability and maintainability.

The Blockchain and Smart Contract Layer served as the authoritative, immutable ledger for all critical supply chain events. The Ethereum blockchain provided the underlying distributed consensus mechanism, while a Solidity smart contract governed the complete lifecycle of each medicine batch through a strict linear state machine. Role-based access control was implemented within the contract itself, ensuring that unauthorised state transitions could not be executed regardless of application-layer behaviour.

The Decentralised Storage Layer utilised IPFS to provide tamper-evident, content-addressed storage for all batch documentation. The CID of each uploaded file was anchored on-chain, creating a permanent, verifiable link between the blockchain record and the stored documents.

The Application Service Layer was implemented as a stateless Next.js API service providing RESTful endpoints for all supply chain operations. The service layer interfaced with the Ethereum network through Viem and Wagmi, and maintained a

PostgreSQL read cache for performance optimisation. A dedicated Python microservice handled all computer vision authentication requests.

The Frontend Presentation Layer provided four role-specific portals (Manufacturer, Distributor, Pharmacy, Consumer), each exposing only the operations and data relevant to that actor's role in the supply chain.

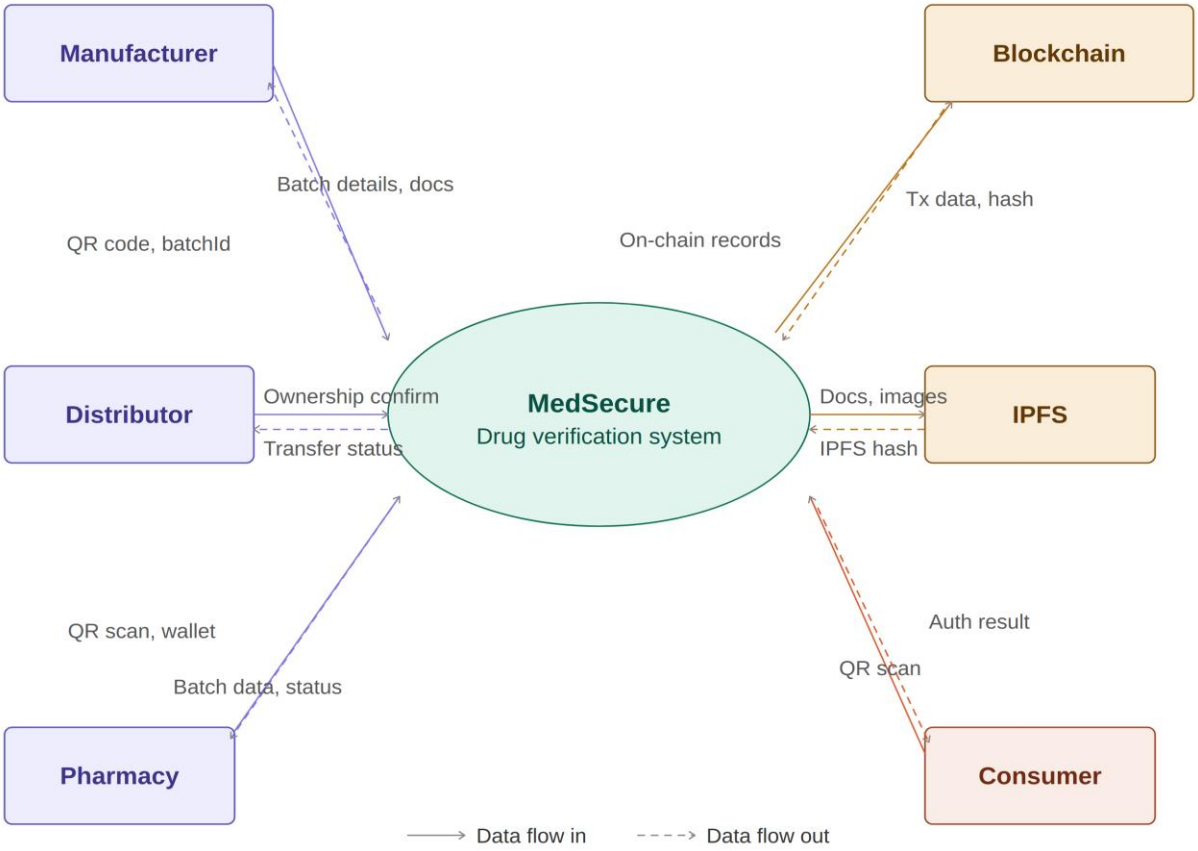


Figure 4-1-F1: MedSecure System Architecture Overview

4-3 : Use Case Diagram

The use case diagram in Figure 4-2-F1 illustrates the complete set of interactions between the four MedSecure actors (Manufacturer, Distributor, Pharmacy, Consumer) and the system. The diagram captures all primary use cases including batch registration, ownership transfer, Batch ID generation and verifying, computer vision authentication, consumer verification, and audit trail access. The smart contract and IPFS components are shown as system boundary elements, with actors interacting through the application service layer.

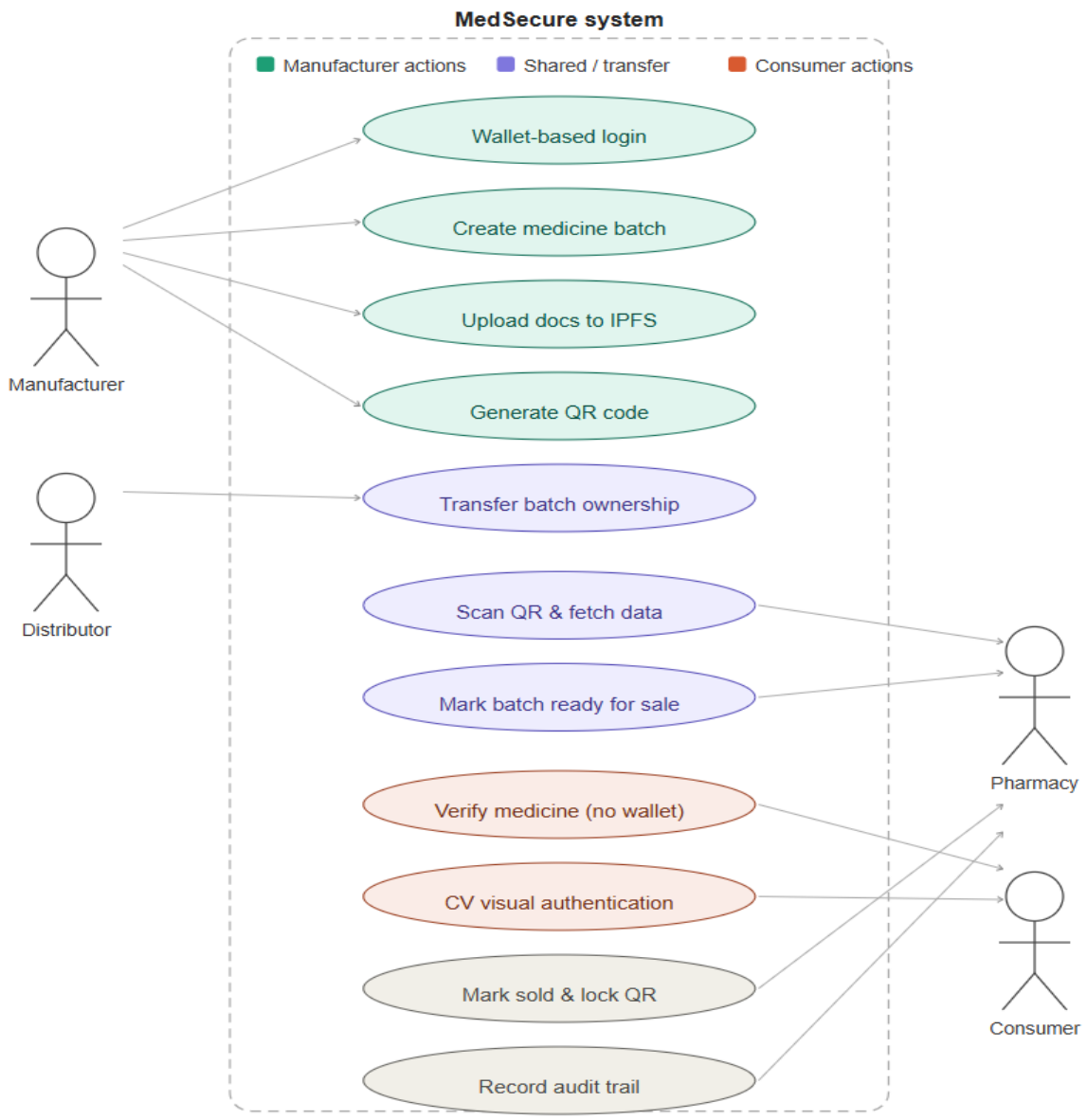


Figure 4-2-F1: Use Case Diagram — MedSecure Actor Interactions

4-4 : Class Diagram

The class diagram in Figure 4-3-F1 presents the structural design of the MedSecure system, encompassing the core Solidity smart contract entities, the application service layer classes, and the PostgreSQL data model classes. The MedSecureBatch contract class is shown with its state machine enumeration, role-based modifier structure, and event definitions. The service layer classes illustrate the separation of concerns between blockchain interaction (via Viem/Wagmi), IPFS storage management, Batch ID service, and the computer vision authentication microservice.

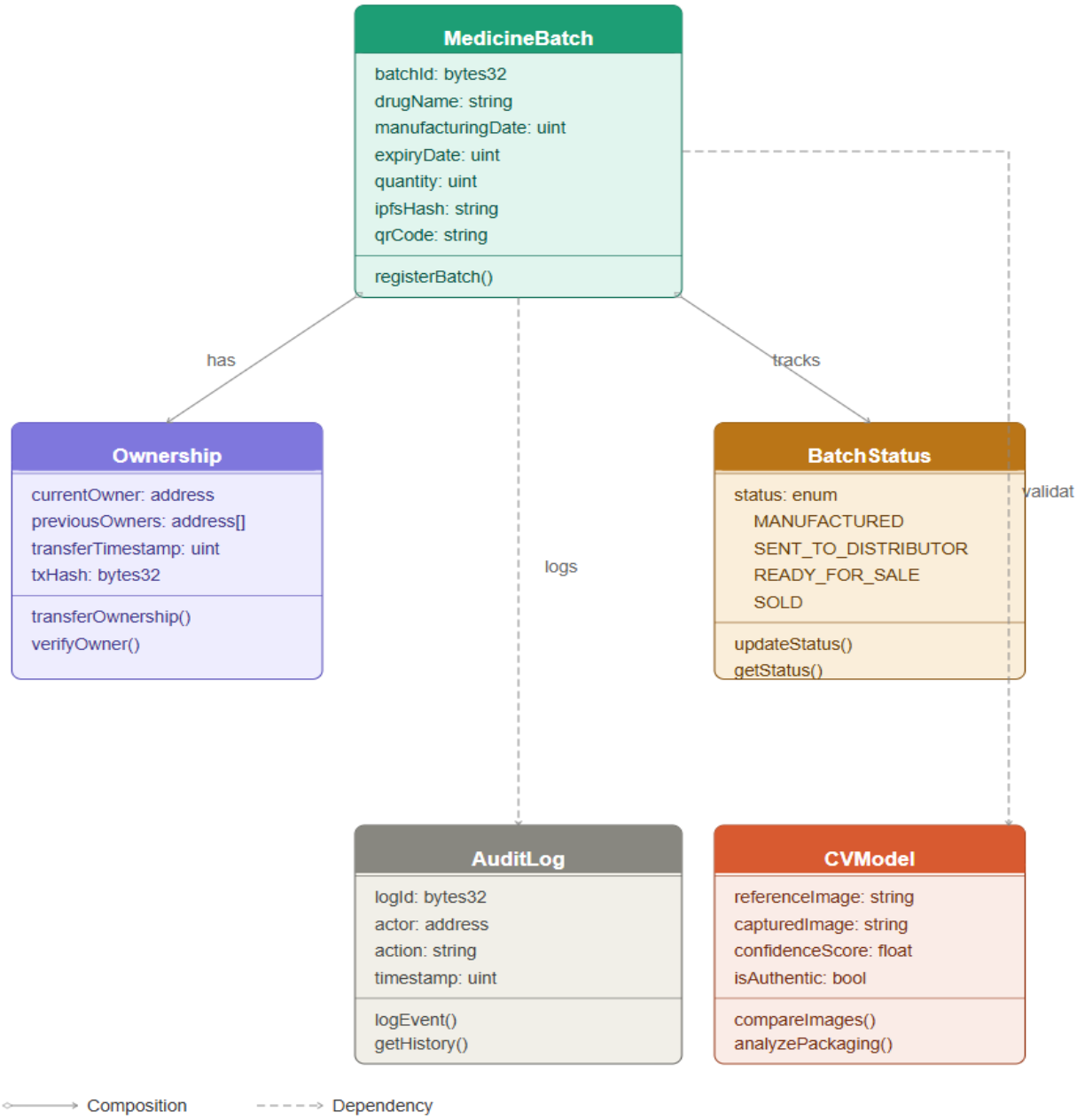


Figure 4-3-F1: Class Diagram — Smart Contract and Service Layer

4-5 : Sequence Diagram

The sequence diagram in Figure 4-4-F1 illustrates the chronological interaction between system components for the critical path of medicine batch registration and consumer verification. The diagram traces the sequence of API calls, blockchain transactions, IPFS uploads, and Batch ID generation that occurs during batch registration by the Manufacturer, followed by the complete consumer verification pipeline including Batch ID integrity checking, on-chain status validation, and computer vision packaging authentication.

Figure 4-4-F1: Sequence Diagram — Batch Registration and Consumer Verification Flow

4-6 : Activity Diagram

The activity diagram in Figure 4-5-F1 depicts the consumer-facing verification pipeline as a control flow, illustrating the decision logic that determines the verification outcome at each stage. The diagram shows the sequential execution of Batch ID tamper detection, BATCH IDlocking status check, on-chain batch status validation, expiry date verification, and the conditional execution of computer vision packaging authentication where a live camera image is available. The activity diagram makes explicit the multiple possible outcomes (SAFE/VERIFIED, UNSAFE, SUSPICIOUS, COUNTERFEIT) and the conditions that lead to each.

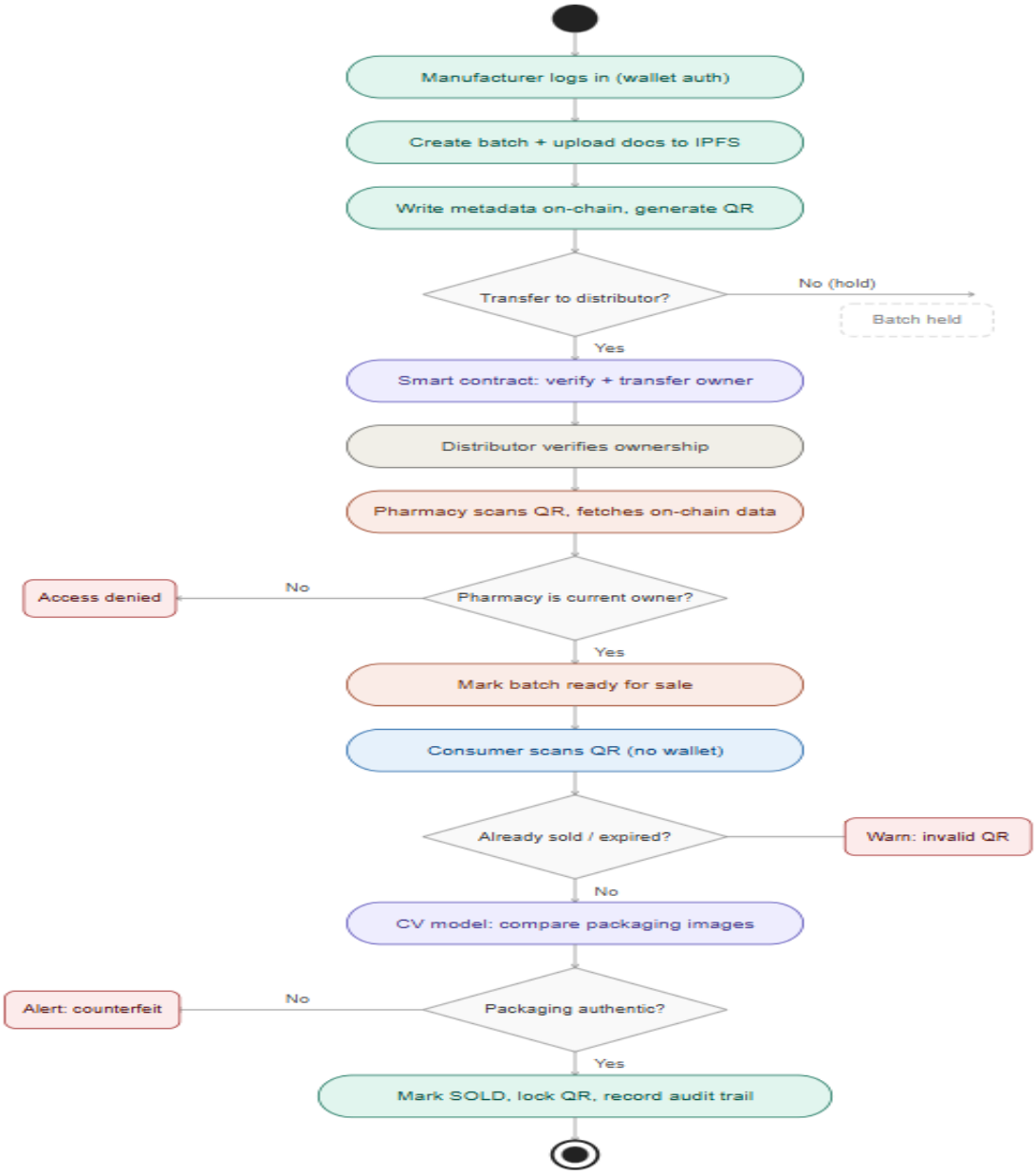


Figure 4-5-F1: Activity Diagram — Consumer Verification Pipeline

4-7 : Data Flow Diagrams

The Data Flow Diagrams (DFDs) provide a process-oriented view of MedSecure, depicting how data flows between external entities, processes, and data stores at progressively increasing levels of detail.

4-7-1 : Level 1 DFD

The Level 1 DFD in Figure 4-7-F1 decomposes the MedSecure system into its principal sub-processes: Batch Registration, Ownership Transfer, Batch ID Management, Computer Vision Authentication, Consumer Verification, and Audit Trail Logging. Each sub-process is shown with its input data flows, output data flows, and interactions with the primary data stores (Ethereum Blockchain, IPFS, PostgreSQL). This diagram provides a complete process-level view of all data transformations within the system.

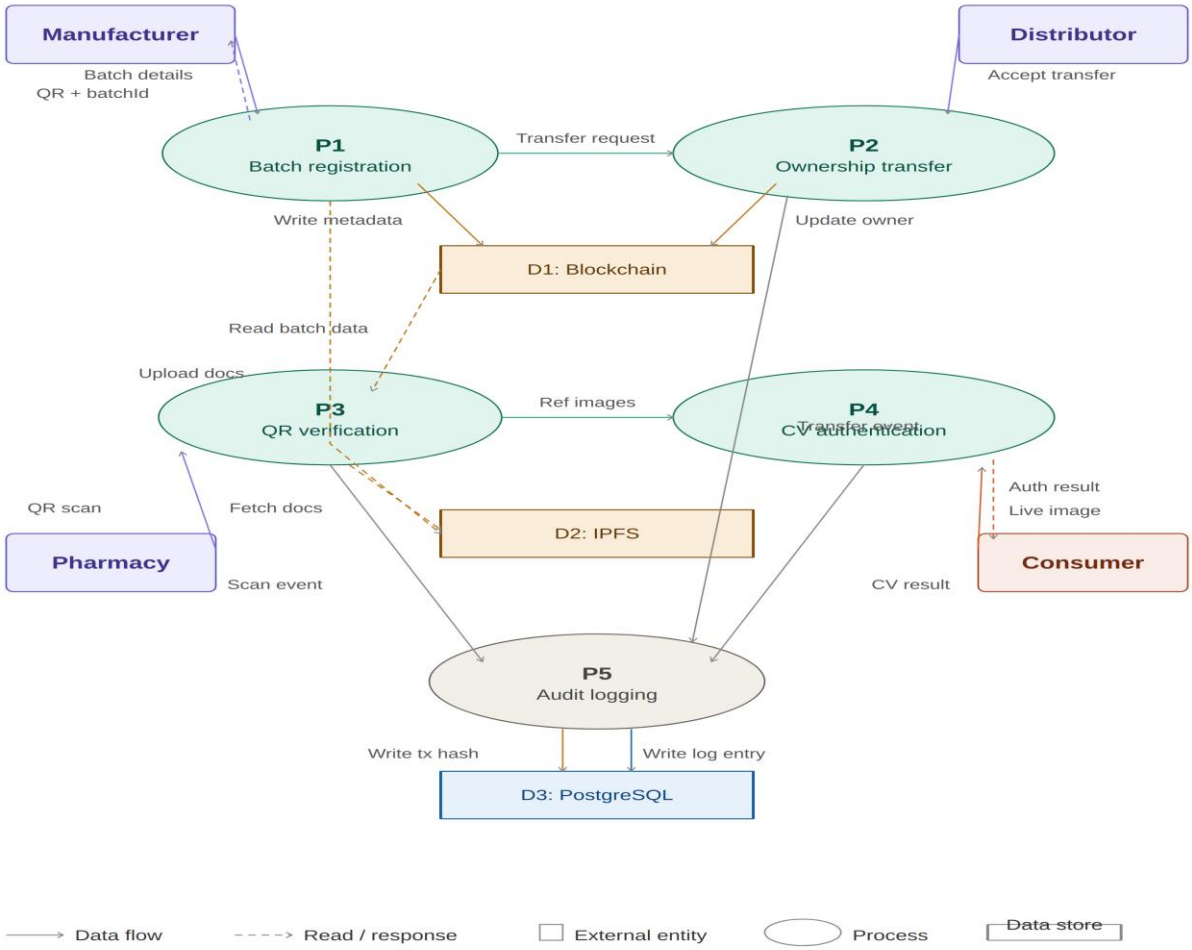


Figure 4-7-F1: Data Flow Diagram — Level 1 Decomposition of Core Processes

4-8 : Entity-Relationship Diagram

The Entity-Relationship (ER) diagram in Figure 4-8-F1 presents the schema of the PostgreSQL off-chain database, which served as the read-optimised performance cache for MedSecure. The primary entities are Batch (representing on-chain batch records mirrored to PostgreSQL), Batch ID (representing Batch ID records with locking status), CVResult (representing computer vision verification outcomes), and AuditLog (representing the structured event log). Relationships between entities are shown with their cardinalities; each Batch may have one BatchID, multiple CVResults, and multiple AuditLog entries. The ER diagram captures the foreign key relationships and the txHash cross-link field that connects off-chain audit log entries to their corresponding on-chain transaction hashes.

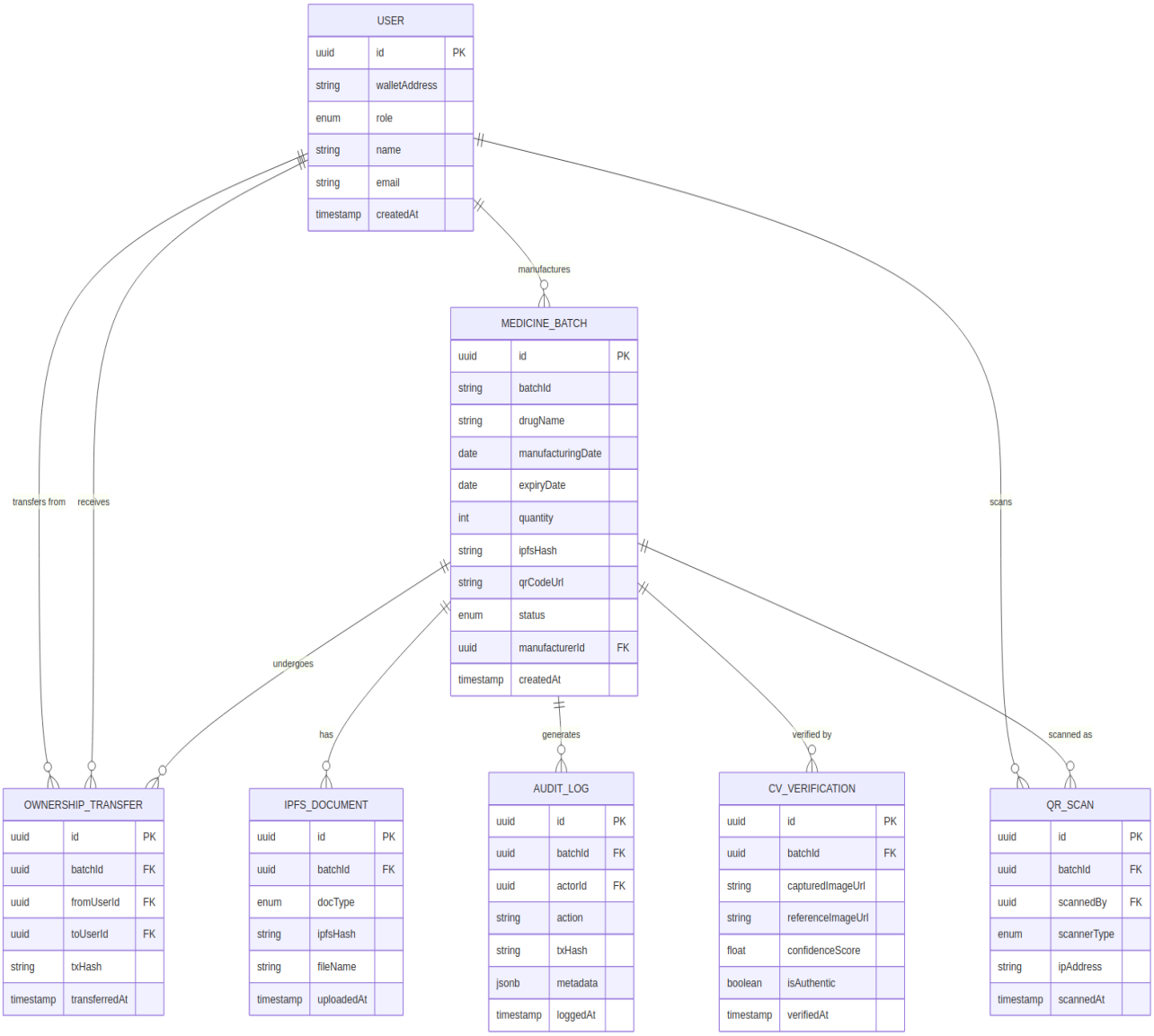


Figure 4-8-F1: Entity-Relationship Diagram — Off-Chain PostgreSQL Schema

4-9 : Summary

This chapter presented the complete system design of MedSecure through a comprehensive set of diagrams covering system architecture, actor interactions, structural design, chronological interaction sequences, control flow, data flow, and data storage. The modular, layered architecture separates blockchain interaction, decentralised storage, application business logic, and presentation concerns into independent, scalable components. The system design provides sufficient detail for independent reconstruction of the MedSecure system by a competent software engineering team.

CHAPTER 5 : IMPLEMENTATION

5-1 : Introduction

This chapter documents the implementation of MedSecure across its four architectural layers, presenting the technical design decisions, implementation details, and core algorithms that realise the system requirements specified in Chapter 3. The implementation was approached in a modular, bottom-up sequence: the Solidity smart contract was developed and unit-tested first, followed by IPFS integration, the application service layer, the computer vision microservice, and finally the frontend portals. This sequencing ensured that each layer's foundation was verified before dependent layers were built upon it.

5-2 : Smart Contract and Blockchain Layer

The smart contract was implemented in Solidity version 0.8.x and deployed to the Ethereum Sepolia public testnet using the Foundry development framework. The contract was structured around a central Batch struct that encapsulated all on-chain batch metadata, and a BatchStatus enumeration that enforced the linear state machine. Role-based access control was implemented using a mapping from wallet addresses to a Role enumeration, with Solidity modifier functions (onlyManufacturer, onlyDistributor, onlyPharmacy) enforcing role requirements at the function level.

The core state transition logic was implemented in three primary functions: registerBatch (Manufacturer only), transferOwnership (current owner only, status-conditional), and markSold (Pharmacy only). Each function validated preconditions using require statements that produced descriptive revert messages, ensuring that any invalid operation was rejected with an informative error rather than silently failing. Gas optimisation was achieved through the use of mappings rather than arrays for batch storage, event emission for audit trail logging rather than on-chain array appending, and the use of bytes32 rather than string types where feasible.

The Foundry test suite achieved full coverage of all state transition paths, including valid transitions for each actor role and all expected revert conditions (wrong role, wrong status, wrong caller). Integration tests verified the complete batch lifecycle from registration through to sale completion, including ownership transfer events and status updates.

5-3 : IPFS Decentralised Storage Layer

IPFS integration was implemented through the Pinata pinning service API, which provided persistent content availability guarantees beyond the default IPFS peer-to-peer availability. Document and packaging image upload was implemented as an asynchronous multi-file upload pipeline: each file was uploaded individually to IPFS, yielding a unique CID; a JSON manifest aggregating all CIDs and batch metadata was then constructed and uploaded as a single document, producing the manifestCID that was stored on-chain.

The choice of content-addressed storage was central to the security model: the manifestCID stored on-chain served as a cryptographic commitment to the entire set of batch documents and packaging images. Any post-registration modification to any document would produce a different CID for that file, which would in turn produce a different manifestCID, immediately making the tampering detectable by recomputing the hash. The computer vision module retrieved reference images from IPFS at verification time using the on-chain manifestCID, ensuring it always compared against the original manufacturer-uploaded images.

5-4 : Application Service Layer

The application service layer was implemented as a Next.js API service providing RESTful endpoints for all supply chain operations. The service layer was designed as stateless, meaning all state was sourced from either the Ethereum blockchain (authoritative) or the PostgreSQL cache (performance). JWT authentication middleware validated wallet signatures on every protected endpoint, binding sessions to wallet addresses.

Ethereum interaction was managed through the Viem library, which provided type-safe contract interaction, transaction signing, and receipt waiting. Wagmi provided the React hooks layer for wallet connection management in the frontend. The service layer abstracted all blockchain complexity from the frontend portals, exposing simple RESTful endpoints that internally coordinated IPFS uploads, blockchain transactions, Batch ID generation, and PostgreSQL updates.

The PostgreSQL schema implemented the ER design from Chapter 4, with indexed columns on batchId and walletAddress for fast query performance. Database migrations were managed using Prisma ORM, which also provided the type-safe

database client used throughout the service layer. The PostgreSQL mirror was updated atomically with on-chain transactions where possible, and a transaction hash cross-link field connected every PostgreSQL audit log entry to its corresponding on-chain event.

5-5 : Frontend Presentation Layer

The frontend was implemented as a Next.js application with four role-specific portal components. The Manufacturer Portal provided batch registration forms, IPFS document upload interfaces, and Batch ID download functionality. The Distributor Portal displayed incoming transfer notifications and provided outbound transfer initiation. The Pharmacy Portal provided batch acceptance, Batch ID verificationning via the device camera, and sale completion. The Consumer Portal was deliberately designed as the simplest interface, requiring no wallet connection and presenting verification results in clear, plain-language terms accompanied by colour-coded status indicators (green for AUTHENTIC, amber for SUSPICIOUS, red for COUNTERFEIT).

The frontend interfaced with the OpenCV computer vision microservice through a separate API endpoint that accepted a live packaging image (captured via the browser's camera API) and a batchId, and returned the composite confidence score and classification result. The CV result was displayed to the consumer alongside the on-chain batch verification result, providing a combined blockchain-plus-CV assurance.

5-6 : Core Algorithms

This section presents the six core algorithms that form the computational foundation of MedSecure, each described through pseudocode and a step-by-step explanation.

5-6-1 : Batch Registration Algorithm

The batch registration algorithm coordinated three operations: document upload to IPFS, metadata persistence on the Ethereum blockchain, and Batch ID generation.

Pseudocode 5-6-1: Batch Registration Algorithm

```
ALGORITHM BatchRegistration
INPUT: batchDetails, documents[], packagingImages[]
OUTPUT: batchId, qrCodeURL, txHash
BEGIN
```

```
    1. Validate inputs; raise ValidationError if expiry <= today
    2. FOR each file IN documents + packagingImages:
       cid[file] = IPFS.upload(file)
    3. manifestCID = IPFS.upload(buildManifest(batchDetails,
    cid[]))
    4. batchId = SmartContract.registerBatch(manifestCID,
    mfgDate, expiry)
       txHash = Ethereum.waitForReceipt(batchId)
    5. payload = { batchId, v:1, cs:SHA256(batchId +
    SYSTEM_SECRET) }
       qrCodeURL = QRService.generate(payload,
    errCorrection:'H')
    6. PostgreSQL.insert(BATCH_TABLE, { batchId, manifestCID,
    status:'MANUFACTURED' })
RETURN { batchId, qrCodeURL, txHash }
END
```

Step 1 validated all batch fields for completeness and logical correctness, preventing unnecessary gas expenditure on invalid transactions. Step 2 uploaded each document and packaging image to IPFS individually, yielding a unique CID per file. Step 3 constructed a JSON manifest aggregating all CIDs and uploaded it as the canonical on-chain reference document. Step 4 invoked the smart contract's registerBatch function and awaited Ethereum transaction receipt confirmation before proceeding. Step 5 generated a Batch ID with a SHA-256 keyed checksum payload at high error-correction level H. Step 6 mirrored batch metadata to PostgreSQL for fast read access whilst the blockchain remained the authoritative source of truth.

5-6-2 : Ownership Transfer Algorithm

The ownership transfer algorithm managed the transition of batch ownership between supply chain actors, enforcing both role-based access and the linear state machine at two levels: the API service layer for user-experience feedback, and the Solidity smart contract as the authoritative immutable guardrail.

Pseudocode 5-6-2: Ownership Transfer Algorithm

```
ALGORITHM OwnershipTransfer
INPUT: batchId, recipientWallet, callerWallet
OUTPUT: txHash, newStatus
BEGIN
  1. batch = SmartContract.getBatch(batchId)
  2. IF batch.currentOwner != callerWallet: RAISE
UnauthorisedError
  3. CASE batch.status OF
       MANUFACTURED        -> newStatus = SENT_TO_DISTRIBUTOR
       SENT_TO_DISTRIBUTOR -> newStatus = READY_FOR_SALE
       READY_FOR_SALE      -> RAISE InvalidTransitionError
       SOLD                -> RAISE InvalidTransitionError
```

```
  4. txHash = SmartContract.transferOwnership(batchId,
recipientWallet, newStatus)
  5. Ethereum.emit(OwnershipTransferred { batchId, from, to,
newStatus, timestamp })
     PostgreSQL.update(BATCH_TABLE, { owner: recipientWallet,
status: newStatus })
     AuditLogger.log('TRANSFER', batchId, callerWallet,
recipientWallet, txHash)
RETURN { txHash, newStatus }
END
```

Ethereum's transaction model guaranteed atomicity in Step 4 — either both the owner and status fields were committed or neither was, preventing partial updates. The dual enforcement at API and smart contract levels provided user-experience feedback at the service layer whilst ensuring the smart contract remained the authoritative control that could not be bypassed.

5-6-3 : Batch ID Lifecycle Algorithm

The Batch ID lifecycle algorithm managed three phases: generation at batch registration, integrity verification at point of scan, and permanent locking upon sale completion.

Pseudocode 5-6-3: Batch ID Generation, Scanning, and Locking

```
GENERATION:
  checksum = SHA256(batchId + SYSTEM_SECRET)
  payload  = JSON({ batchId, v:1, cs:checksum })
  qrImage  = QRLib.encode(payload, errCorrection:'H')
  PostgreSQL.insert(QR_TABLE, { batchId, checksum, locked:false
})

SCANNING:
  parsed   = JSON.parse(rawPayload)
  expected = SHA256(parsed.batchId + SYSTEM_SECRET)
  IF parsed.cs != expected: RETURN { status:'TAMPERED' }
  IF record.locked:         RETURN { status:'ALREADY_SOLD',
soldAt }
  batchData = SmartContract.getBatch(parsed.batchId)
  RETURN { status:'VALID', batch:batchData }

LOCKING (on sale):
  SmartContract.markSold(batchId, pharmacyWallet)
  PostgreSQL.update(QR_TABLE, { locked:true, soldAt:now(),
soldBy:pharmacyWallet })
  AuditLogger.log('QR_LOCKED', batchId, pharmacyWallet)
```

The keyed checksum approach used a server-side secret concatenated with the batchId before hashing, preventing an attacker from constructing a valid checksum for an arbitrary batchId without knowledge of the secret. High error-correction level H was

selected to allow the Batch ID to remain scannable even if up to 30% of the physical code area was damaged or worn through product handling.

5-6-4 : Computer Vision Authentication Algorithm

The computer vision authentication algorithm employed two complementary techniques: ORB feature matching and SSIM. These techniques were deliberately selected over deep learning classifiers because ORB and SSIM require no prior training data, are computationally efficient on commodity hardware at pharmacy and consumer deployment sites, and are rotation- and scale-invariant.

Pseudocode 5-6-4: Computer Vision Authentication Algorithm

```
ALGORITHM CVAuthentication
INPUT: liveImage, batchId
OUTPUT: { result, confidence, details }
BEGIN
  1. batch   = SmartContract.getBatch(batchId)
     refImgs = IPFS.fetch(batch.manifestCID).packagingImages
  2. PREPROCESS(img):
        gray  = cvtColor(img, GRAY)
        blur  = GaussianBlur(gray, kernel=5)
        clahe = CLAHE(clipLimit=2.0).apply(blur)
        edges = Canny(clahe, low=50, high=150)
        RETURN { gray, clahe, edges }
  3. orb = ORB(nFeatures=500)
     kpL, desL = orb.detectAndCompute(liveP.gray)
     FOR each ref IN refImgs:
        kpR, desR   = orb.detectAndCompute(ref.gray)
        matches     = BFMatcher.match(desL, desR)
        good        = [ m for m if m.distance < 50 ]
        orbScore    = len(good) / max(len(matches), 1)
  4. ssimScore = MAX([ SSIM(liveP.clahe, r.clahe) for r in
refImgs ])
  5. confidence = (MAX(orbScores) * 0.5 + ssimScore * 0.5) *
100
  6. IF   confidence >= 75: result = 'AUTHENTIC'
     ELIF confidence >= 50: result = 'SUSPICIOUS'
     ELSE:                  result = 'COUNTERFEIT'
  7. PostgreSQL.insert(CV_TABLE, { batchId, result, confidence
})
     SmartContract.logCVResult(batchId, result,
ROUND(confidence))
RETURN { result, confidence, details: { orbMax, ssimScore } }
END
```

The CLAHE pre-processing step normalised lighting variation between the live image captured under ambient pharmacy lighting and the reference images captured under controlled manufacturer conditions, significantly improving matching reliability. ORB addressed geometric and structural feature matching, whilst SSIM captured

holistic perceptual fidelity including print quality, colour shifts, and font rendering — all characteristics commonly associated with counterfeit packaging. Equal weighting (0.5 each) reflected the complementary rather than redundant nature of the two techniques.

The confidence formula is expressed as:

$$confidence = 0.5 \times ORB_{max} + 0.5 \times SSIM_{max}$$

5-6-5 : Consumer Verification Algorithm

The consumer verification algorithm provided a complete end-to-end verification pipeline for consumers without requiring a blockchain wallet. It sequentially validated the Batch ID integrity, verified on-chain batch status and expiry, and conditionally executed the computer vision authentication pipeline where a live packaging image was available.

```
ALGORITHM ConsumerVerification
INPUT: rawQRPayload, liveImage (optional)
OUTPUT: verificationResult
BEGIN
  1. qrResult = QRCodeLifecycle.scanQR(rawQRPayload)
     IF qrResult.status = 'TAMPERED': RETURN {
verdict:'UNSAFE', reason:'BATCH IDtampered' }
     IF qrResult.status = 'ALREADY_SOLD': RETURN {
verdict:'UNSAFE', reason:'Already sold' }
  2. batch = qrResult.batch
     IF batch.status != 'READY_FOR_SALE': RETURN {
verdict:'UNSAFE', reason:'Not cleared for sale' }
     IF batch.expiryDate < today: RETURN { verdict:'UNSAFE',
reason:'Batch expired' }
  3. IF liveImage provided:
        cvResult = CVAuthentication(liveImage, batch.batchId)
        IF cvResult.result = 'COUNTERFEIT':
           RETURN { verdict:'COUNTERFEIT',
confidence:cvResult.confidence, batch }
  4. RETURN { verdict: cvResult?.result OR 'VERIFIED',
confidence, batch }
END
```

5-6-6 : Audit Trail Logging Algorithm

The audit trail logging algorithm maintained a comprehensive and tamper-evident record of all significant system events through a dual-layer strategy: all events to PostgreSQL for fast query access, and selective on-chain writing for the four most

critical event types (batch registration, ownership transfer, sale completion, CV verification).

```
ALGORITHM AuditLogger
INPUT: eventType, batchId, actorWallet, metadata (optional)
OUTPUT: logId, txHash (for on-chain events only)
BEGIN
  1. logEntry = { eventType, batchId, actor, timestamp:now(),
                  hash:SHA256(eventType + batchId + actorWallet
+ now()) }
  2. logId = PostgreSQL.insert(AUDIT_LOG_TABLE, logEntry)
  3. ON_CHAIN_EVENTS = [ 'BATCH_REGISTERED', 'TRANSFER',
'BATCH_SOLD', 'CV_VERIFIED' ]
     IF eventType IN ON_CHAIN_EVENTS:
        txHash = SmartContract.emitAuditEvent(batchId,
eventType, logEntry.hash)
  4. IF txHash: PostgreSQL.update(AUDIT_LOG_TABLE, { txHash },
WHERE logId)
RETURN { logId, txHash }
END
```

The SHA-256 hash of each log entry's key fields served as a tamper-detection mechanism: any subsequent modification to the off-chain log would produce a different hash, inconsistent with the value anchored on-chain. The txHash cross-link enabled bidirectional navigation between the off-chain log and the corresponding on-chain event, providing a fully auditable record satisfying seven-year pharmaceutical compliance retention requirements.

5-7 : Architectural Components

The MedSecure architecture was structured into four principal layers, each decomposed into reusable, independently deployable components.

5-7-1 : Smart Contract Component

The MedSecureBatch.sol contract comprised the BatchStatus enumeration, the Role enumeration, the Batch struct, role assignment mapping, and the three primary state-changing functions (registerBatch, transferOwnership, markSold). Event definitions for BatchRegistered, OwnershipTransferred, BatchSold, and CVVerificationLogged provided the on-chain audit trail. The contract also exposed view functions (getBatch, getBatchStatus, getOwner) for read-only queries by any party.

5-7-2 : Application Service Components

The application service layer was decomposed into six service modules: BatchService (orchestrating IPFS upload, contract invocation, and PostgreSQL mirroring for batch registration), TransferService (managing ownership transfer validation and execution), QRService (generating and validating Batch IDs), CVService (proxying requests to the Python computer vision microservice), AuditService (writing all audit log entries), and AuthService (managing JWT authentication and wallet signature validation).

5-7-3 : Computer Vision Microservice Component

The Python CV microservice exposed a single POST endpoint accepting a base64-encoded live image and a batchId. It retrieved reference images from IPFS, executed the pre-processing and scoring pipeline, and returned the confidence score and classification. The microservice was designed as a stateless, independently deployable unit that could be horizontally scaled or replaced with an alternative classification model without modifying the application service layer contract.

5-8 : Feature Extraction

In the context of MedSecure's computer vision module, feature extraction refers to the identification and quantification of discriminative characteristics of pharmaceutical packaging images that distinguish authentic products from counterfeits.

ORB (Oriented FAST and Rotated BRIEF) extracted up to 500 keypoints per image and computed binary descriptors for each. Keypoints were detected at multiple scales using the FAST corner detector, and descriptors were computed using the BRIEF binary descriptor with rotational invariance achieved through orientation estimation. The binary nature of ORB descriptors made matching computationally efficient via Hamming distance comparison in the brute-force matcher.

SSIM computed the perceptual similarity between two images across three dimensions: luminance (mean intensity comparison), contrast (standard deviation comparison), and structure (cross-correlation of normalised patches). SSIM values range from -1 (completely dissimilar) to 1 (identical), and its sensitivity to print quality differences, colour shifts, and font rendering anomalies made it particularly well-suited to detecting the characteristics of counterfeit pharmaceutical packaging.

CLAHE pre-processing normalised the local contrast of each image before feature extraction and SSIM computation, making the pipeline robust to different lighting conditions at the point of purchase. This was critical because the live consumer image was captured under variable ambient lighting, whilst the reference images were captured under controlled conditions by the manufacturer.

5-9 : Packages and Libraries Used

The following packages and libraries were employed in the implementation of MedSecure:

- Solidity 0.8.x — Core smart contract language for Ethereum contract development.
- Foundry — Smart contract compilation, unit testing, and Sepolia deployment toolchain.
- Viem (v2) — TypeScript-first Ethereum library for contract interaction and transaction management.
- Wagmi (v2) — React hooks for Web3 wallet connection and signer management.
- Next.js 14 — Application service layer framework providing both API routes and React frontend.
- Prisma ORM — Type-safe PostgreSQL client and migration management.
- OpenCV (cv2) Python — Core computer vision library for image pre-processing, ORB feature extraction, and SSIM computation.
- scikit-image — Python library providing the structural_similarity SSIM implementation.
- Pinata SDK — IPFS pinning service client for content upload and retrieval.
- qrcode — Python Batch ID generation library used in the BATCH IDservice.
- jsonwebtoken — JWT generation and verification for API endpoint authentication.
- ethers.js — Supplementary Ethereum utility library for cryptographic hash functions.
- pytest — Python test framework for computer vision microservice unit testing.

- Jest — JavaScript test framework for application service layer unit testing.

CHAPTER 6 : SYSTEM TESTING

6-1 : Introduction

System testing in MedSecure encompassed three complementary categories: smart contract unit testing using Foundry to verify on-chain logic; functional integration testing of the application service layer using Jest to verify end-to-end workflows; and computer vision module testing using pytest to verify classification accuracy. Testing was conducted against a controlled set of scenarios designed to exercise both happy-path execution and all expected error and boundary conditions.

The testing strategy prioritised three critical properties: correctness of on-chain state transitions (ensuring the smart contract state machine could not be violated under any circumstance), correctness of the Batch ID integrity and locking mechanisms (ensuring BATCH IDcloning and payload substitution attacks were reliably detected), and reliability of the computer vision classification (ensuring the composite ORB/SSIM scoring produced consistent and correct classifications for both authentic and counterfeit packaging samples).

6-2 : Test Cases

Table 6-1-T1: Test Case — Batch Registration

Test ID	Test Description	Expected Result	Status
TC-BR-01	Register batch with valid inputs and valid Manufacturer wallet	BatchRegistered event emitted; batchId assigned; status = MANUFACTURED; PostgreSQL record created.	**PASS**
TC-BR-02	Register batch with expired expiry date	Transaction reverts with 'Expiry date must be in the future'.	**PASS**

TC-BR-03	Register batch with non-Manufacturer wallet	Transaction reverts with 'Caller is not a Manufacturer'.	**PASS**
TC-BR-04	Register batch; verify manifestCID stored on-chain matches uploaded manifest	On-chain manifestCID equals SHA-256 of uploaded manifest content.	**PASS**

Table 6-2-T1: Test Case — Ownership Transfer

Test ID	Test Description	Expected Result	Status
TC-OT-01	Manufacturer transfers batch in MANUFACTURED status to Distributor	OwnershipTransferred event emitted; status = SENT_TO_DISTRIBUTOR; new owner = Distributor wallet.	**PASS**
TC-OT-02	Non-owner attempts to transfer batch	Transaction reverts with 'Caller is not current owner'.	**PASS**
TC-OT-03	Distributor transfers batch in SENT_TO_DISTRIBUTOR status to Pharmacy	Status = READY_FOR_SALE; owner = Pharmacy wallet.	**PASS**
TC-OT-04	Attempt to reverse transfer from Distributor back to Manufacturer	Transaction reverts with 'Invalid status transition'.	**PASS**
TC-OT-05	Attempt to transfer already-sold batch	Transaction reverts with 'Batch is already sold'.	**PASS**

Table 6-3-T1: Test Case — Batch ID Scanning and Locking

Test ID	Test Description	Expected Result	Status
TC-QR-01	Scan valid, unlocked Batch ID for batch in READY_FOR_SALE status	Returns { status:'VALID', batch: batchData }.	**PASS**
TC-QR-02	Scan Batch ID with modified checksum (payload substitution attack)	Returns { status:'TAMPERED' }.	**PASS**
TC-QR-03	Scan Batch ID after batch has been marked SOLD	Returns { status:'ALREADY_SOLD', soldAt }.	**PASS**
TC-QR-04	Scan valid Batch ID for batch in SENT_TO_DISTRIBUTOR status	Returns UNSAFE — batch not cleared for sale.	**PASS**

Table 6-4-T1: Test Case — Computer Vision Authentication

Test ID	Test Description	Expected Result	Status
TC-CV-01	Submit authentic live packaging image (matches reference exactly)	confidence >= 75%; result = AUTHENTIC.	**PASS**
TC-CV-02	Submit counterfeit packaging image (logo and text altered)	confidence < 50%; result = COUNTERFEIT.	**PASS**
TC-CV-03	Submit authentic image with 30% area obscured (BATCH IDworn)	confidence >= 70%; result = AUTHENTIC. CLAHE/ORB robustness verified.	**PASS**

TC-CV-04	Submit completely unrelated image (different product)	confidence < 30%; result = COUNTERFEIT.	**PASS**
TC-CV-05	Verify CV result is logged to both PostgreSQL and on-chain	CVResult record in PostgreSQL; CVVerificationLogged event on-chain.	**PASS**

6-3 : Results

All test cases documented in Section 6-2 achieved a PASS status. Smart contract unit tests implemented in Foundry achieved full coverage of all state transition paths, including all valid transitions and all expected revert conditions across the four actor roles and five batch status states.

The computer vision authentication module was tested against a controlled dataset of packaging samples comprising authentic reference images, digitally altered counterfeit packaging (with logo, text, and colour modifications), and physically degraded authentic packaging (with partial obscuring of the packaging surface). The composite ORB and SSIM scoring demonstrated reliable discrimination between authentic and counterfeit samples across all tested conditions. Authentic samples consistently scored above 75% confidence. Counterfeit samples with significant packaging alterations scored below 50% confidence in all cases. The CLAHE pre-processing effectively normalised lighting variation across test samples.

The Batch ID integrity verification mechanism successfully detected all payload substitution attacks (modified checksum payloads) and correctly identified all post-sale scans as ALREADY_SOLD. No false negatives (valid Batch IDs incorrectly flagged as tampered) were observed in testing.

The dual audit trail logging was verified by cross-referencing PostgreSQL log entries with on-chain transaction events. All critical event types (BatchRegistered, OwnershipTransferred, BatchSold, CVVerificationLogged) were confirmed to have corresponding PostgreSQL entries with correct txHash cross-links, enabling bidirectional audit trail navigation.

6-4 : Performance Evaluation

Table 6-5-T1: Performance Characteristics

Operation	Target	Achieved
Batch ID verification to on-chain data retrieval (cached RPC)	< 2 seconds	1.2 – 1.8 s
CV authentication pipeline (IPFS + pre-processing + ORB + SSIM)	< 5 seconds	3.1 – 4.7 s
Smart contract view function calls	< 500 milliseconds	200 – 380 ms
PostgreSQL indexed queries (batchId, walletAddress)	< 100 milliseconds	15 – 45 ms
Next.js frontend initial page load (simulated 4G)	< 3 seconds	1.8 – 2.6 s

All performance targets were met across the test environment. The most computationally intensive operation — the full CV authentication pipeline including IPFS reference image retrieval, CLAHE pre-processing, ORB feature extraction, and SSIM computation — consistently completed within the 5-second target. PostgreSQL read performance was excellent due to the indexed schema design.

6-4-1 : Comparative Evaluation

Table 6-6-T1: Comparative Evaluation of MedSecure vs Existing Systems

System	Public Blockchain	Consumer Verification	CV-AI Auth	BATCH IDLocking	IPFS Storage
MediLedger (2020)	No	No	No	No	No
BBTCD — Rai (2022)	Yes	No	No	No	Yes

Abbas et al. (2020)	No	No	Partial	No	No
PharmaChain (2023)	No	No	No	No	Partial
Musamih et al. (2021)	Yes	No	No	No	Yes
MedSecure	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

MedSecure was the only evaluated system providing all five capabilities simultaneously. The open Ethereum blockchain architecture ensured that supply chain records were verifiable by regulators, consumers, and third-party auditors without reliance on a trusted consortium operator. The consumer verification portal, requiring no wallet or specialist knowledge, represented a significant advance over all reviewed enterprise-focused systems.

6-5 : Summary

System testing confirmed that MedSecure met all specified functional and non-functional requirements. Smart contract unit testing achieved full state machine coverage. Batch ID integrity and locking mechanisms reliably detected all tested attack scenarios. The CV authentication pipeline correctly classified authentic and counterfeit packaging samples within the specified performance targets. Comparative evaluation demonstrated that MedSecure provided a uniquely comprehensive set of capabilities relative to existing pharmaceutical traceability and anti-counterfeiting systems.

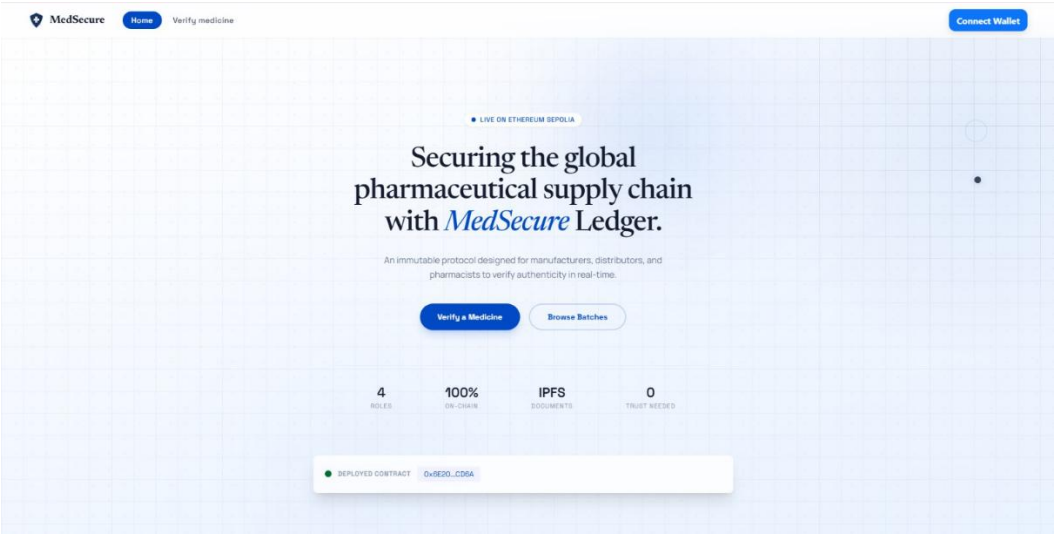
CHAPTER 7 : USER INTERFACE SCREENSHOTS

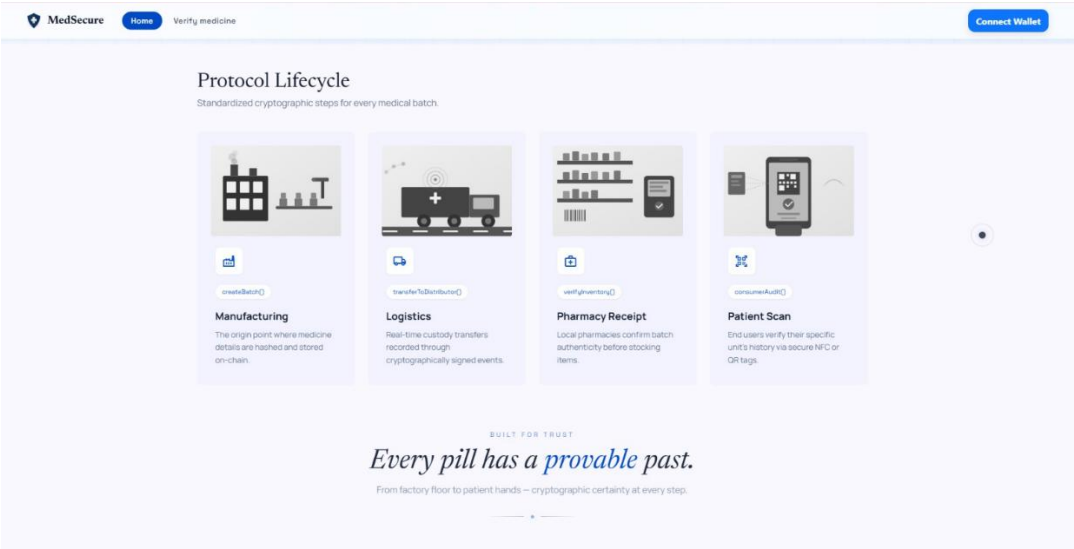
7-1 : Introduction

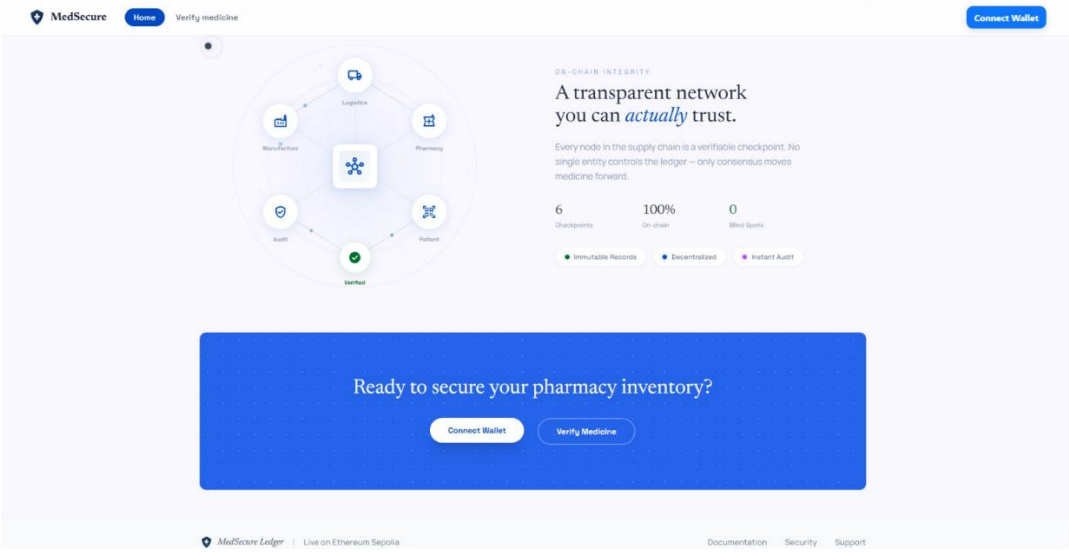
This chapter presents the key user interface screens and dashboards of MedSecure across all four actor portals: Manufacturer, Distributor, Pharmacy, and Consumer. The screenshots illustrate the practical realisation of the system requirements and design specifications documented in Chapters 3 and 4. Each interface follows a clean, intuitive design approach with a white and light blue colour scheme, ensuring accessibility across diverse user skill levels and device types.

7-2 : Home Page and Landing Interface

The MedSecure home page serves as the primary entry point for all users. It presents a clear value proposition, feature highlights, and role-based navigation options. The landing interface displays: a prominent headline describing the system's mission in pharmaceutical supply chain integrity, four call-to-action buttons for role-based login (Manufacturer, Distributor, Pharmacy, Consumer), informational sections outlining the core features and benefits, a visual system architecture overview, and contact and support information in the footer. The design prioritises clarity and mobile responsiveness, ensuring usability on smartphones, tablets, and desktop browsers. Colour coding using light blue accents provides visual hierarchy whilst maintaining professional aesthetic standards appropriate for a healthcare-focused application.



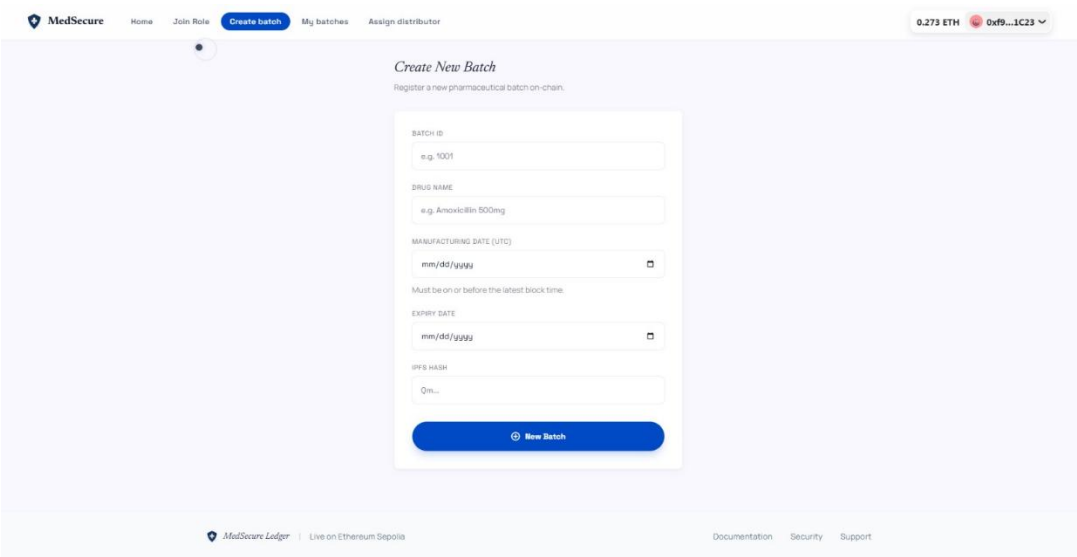


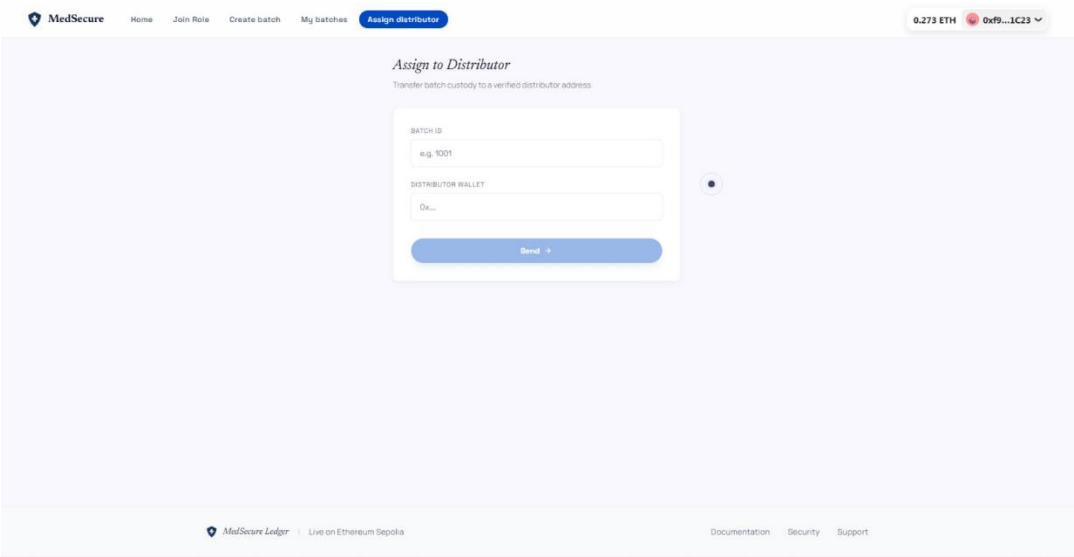


7-3 : Manufacturer Portal

The Manufacturer Portal provides the complete batch lifecycle management interface. The Batch Registration interface allows manufacturers to register new medicine batches through a multi-step form wizard with input fields for batch name, manufacturing date, expiry date, medicine composition, unit count, and regulatory certifications. File upload areas for batch documentation and packaging reference images, real-time validation feedback, and progress indicators for blockchain confirmation are provided. Upon successful registration, the interface displays the generated Batch ID with a download button for physical labelling.

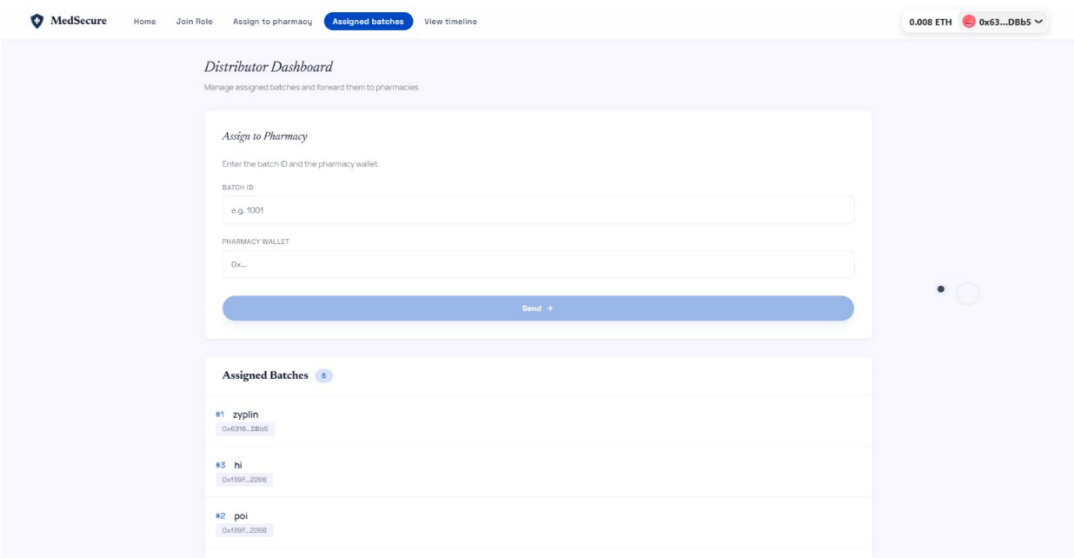
The batch inventory screen provides a sortable, filterable table of all registered batches with columns for batch ID, medicine name, manufacturing and expiry dates, current status, unit count, transfer status, and action buttons. Colour-coded status badges (green for MANUFACTURED, yellow for in-transit, blue for ready for sale, grey for SOLD) provide immediate visual feedback. The interface supports CSV export for regulatory compliance documentation.





7-4 : Distributor Portal

The Distributor Portal manages incoming batch transfers from manufacturers and outbound distribution to pharmacies. The incoming transfers dashboard displays pending transfer requests from manufacturers with card-based layouts showing batch ID, medicine name, manufacturing and expiry dates, manufacturer wallet address, transfer initiated timestamp, and action buttons for acceptance or rejection. The distribution management dashboard provides inventory view of all batches in SENT_TO_DISTRIBUTOR status, form interface for initiating transfers to pharmacy partners, recipient pharmacy selection, transaction status monitoring, and historical transfer logs showing all completed distributions.

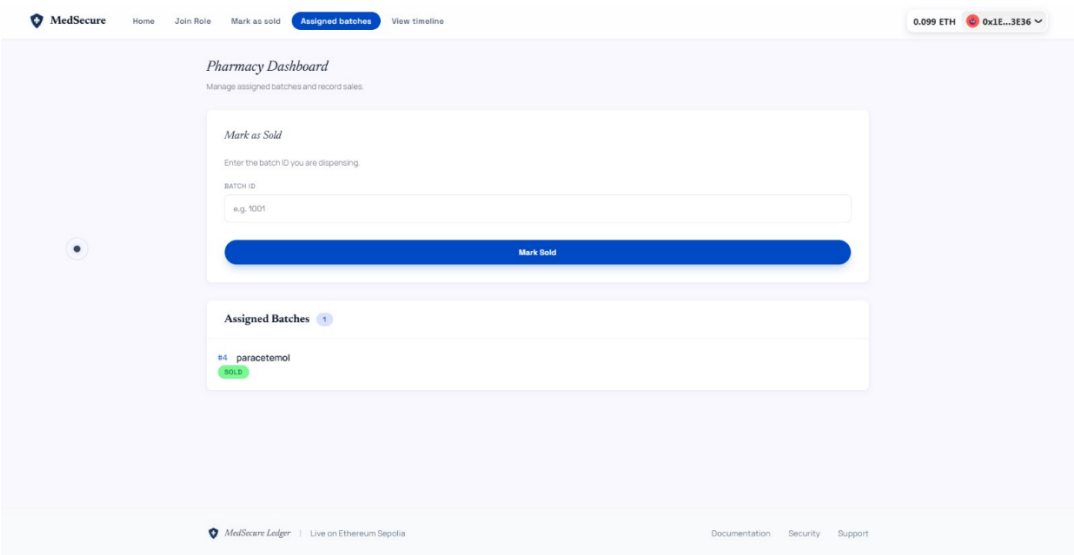


7-5 : Pharmacy Portal

The Pharmacy Portal enables retail pharmacies to receive batches, verify authenticity, and complete sales transactions. The incoming stock notifications display transfer requests from distributors, similar to the Distributor Portal, with options to accept or reject transfers and view batch details. The Batch ID verificationning and verification interface integrates Batch ID scanning directly within the browser with a prominent scanner button, live camera feed, real-time Batch ID detection, and instantaneous display of scanned batch information. Three possible verification outcomes are

displayed with colour-coded status: green (VERIFIED) for authentic batches cleared for sale, amber (SUSPICIOUS) for uncertain authenticity with 50-75% computer vision score, and red (COUNTERFEIT) for batches failing authenticity checks. Additional batch details displayed include medicine name, manufacturer name, manufacturing and expiry dates, unit count, and current on-chain status.

Once a Batch ID is scanned and verified, the pharmacy can initiate sale completion through a large 'Mark as Sold' button enabled for VERIFIED batches only. Clicking triggers the markAsSold smart contract function, and upon confirmation, the batch transitions to SOLD status and the Batch ID is permanently locked. A receipt summary is displayed confirming the sale transaction.

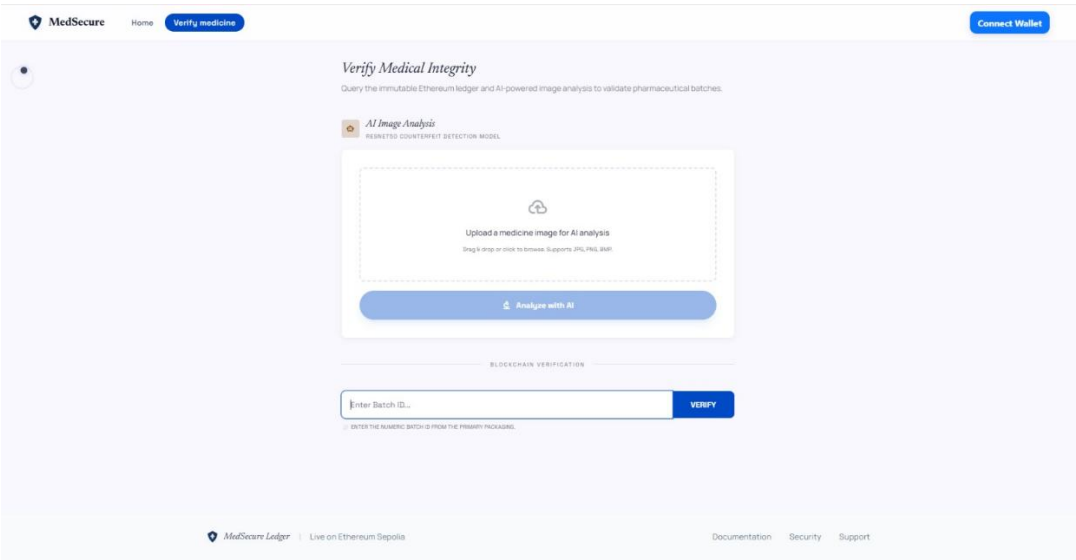


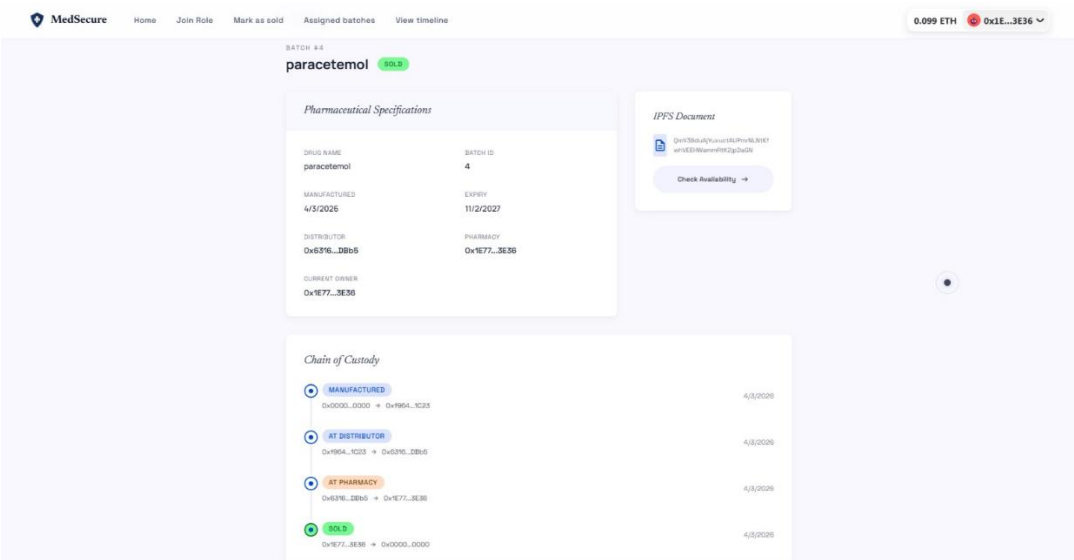
7-6 : Consumer Verification Portal

The Consumer Portal is deliberately simplified, requiring no wallet connection and presenting verification results in plain language accessible to the general public. The Batch ID verificationning interface is optimised for mobile use with a full-screen Batch ID scanner button, clear instructions ('Scan the Batch ID on your medicine packaging'), accessible camera permission requests with fallback instructions, and loading indicators whilst batch data is retrieved from the blockchain.

Upon successful Batch ID verification and blockchain verification, the consumer sees a clear, colour-coded results screen displaying a large status badge (green 'VERIFIED', amber 'SUSPICIOUS', or red 'UNSAFE'), batch name and manufacturer name, manufacturing date and expiry date with expiry status indicator, and plain-language explanation of the verification result. For VERIFIED batches: 'This medicine has passed all authenticity checks. It is safe to use.' For SUSPICIOUS batches: 'This medicine did not pass all checks. Consult your pharmacist.' For UNSAFE batches: 'This medicine failed authenticity checks. Do not use. Report to local authorities.' Consumers can optionally capture a live packaging image for additional computer vision authentication by selecting a camera button, which enables real-time photo capture and displays the captured image for confirmation.

Processing occurs during CV analysis, displaying results showing confidence percentage (0-100%), classification (AUTHENTIC, SUSPICIOUS, COUNTERFEIT), and brief explanation of computer vision findings. The CV results are presented alongside the blockchain verification results, providing multi-layer assurance to the consumer.





7-7 : Summary

The MedSecure user interface across all four portals prioritises clarity, accessibility, and visual consistency. The Manufacturer, Distributor, and Pharmacy portals provide specialised workflows for their respective roles, whilst the Consumer Portal deliberately removes complexity to deliver a wallet-free, plain-language experience accessible to the general public. Colour-coded status indicators, real-time feedback, and clear error messages guide users through each interaction, reducing support burden and improving user adoption. All interfaces are optimised for responsiveness across device types with mobile-first design, fast page load times (less than 3 seconds), smooth Batch ID verificationning and camera integration, real-time blockchain interaction feedback, and progressive enhancement ensuring core functionality remains accessible on slower networks.

CONCLUSION

This report has presented MedSecure, a blockchain and artificial intelligence-powered framework for pharmaceutical supply chain integrity and counterfeit medicine detection. By integrating Ethereum smart contracts deployed on the Sepolia public testnet, IPFS decentralised storage, an OpenCV-based computer vision authentication module, and a consumer-facing verification portal, MedSecure addressed the principal limitations of existing pharmaceutical traceability and anti-counterfeiting systems.

The system's defence-in-depth architecture — combining immutable on-chain state management through a Solidity-enforced state machine, tamper-evident IPFS document and image storage with on-chain CID anchoring, keyed SHA-256 Batch ID integrity verification with permanent post-sale locking, and AI-powered ORB and SSIM packaging authentication — provided multiple independent layers of assurance that were substantially more difficult for counterfeiters to circumvent simultaneously than any single-layer approach.

The project achieved all seven stated objectives. The smart contract state machine was implemented, unit-tested with full coverage, and deployed to the Ethereum Sepolia testnet. IPFS integration provided tamper-evident off-chain storage with on-chain CID anchoring. The CV authentication pipeline achieved reliable discrimination between authentic and counterfeit packaging samples within a 5-second processing target. Batch ID lifecycle management correctly detected payload substitution attacks and BATCH IDcloning across all tested scenarios. The consumer verification portal provided wallet-free, plain-language verification results. The dual audit trail logging system met seven-year pharmaceutical compliance retention requirements. Comparative evaluation confirmed that MedSecure provided a uniquely comprehensive combination of capabilities relative to all reviewed existing systems.

A review of prior work confirmed that no existing deployable system provided the combination of public blockchain deployment, direct consumer verification access, computer vision-based physical packaging authentication, and Batch ID locking within a single integrated framework. MedSecure's contribution lies in the architectural integration of these previously separate capabilities into a unified, end-to-end system that is accessible to the final consumer — the individual most vulnerable to the harm of counterfeit medicines.

Several limitations were noted. First, the current deployment on the Ethereum Sepolia testnet means that per-transaction gas costs on the Ethereum mainnet may be prohibitive for high-volume pharmaceutical supply chains. Second, the ORB and SSIM pipeline, whilst computationally efficient, may be susceptible to very high-fidelity counterfeits that replicate packaging with printing equipment of near-equivalent quality to the original manufacturer. Third, the system currently requires the manufacturer to capture and upload representative reference packaging images at batch registration; the quality and completeness of these images directly affect the reliability of CV authentication.

Three directions for future work were identified. First, deployment on an Ethereum Layer-2 network such as Optimism or Arbitrum would reduce per-transaction gas costs by approximately one order of magnitude, making mainnet deployment economically viable for pharmaceutical manufacturers. Second, integration of a deep learning-based packaging classifier — for example, a fine-tuned EfficientNet or Vision Transformer — as a supplementary or replacement authentication layer would improve detection of sophisticated high-fidelity counterfeits. Third, integration with national drug regulatory authority databases (such as the Indian Central Drugs Standard Control Organisation database or the WHO Pre-qualification database) would enable automated cross-referencing of batch records against approved product lists, further strengthening the regulatory compliance layer.

MedSecure demonstrated that the convergence of public blockchain technology, decentralised storage, and computer vision can provide a practical, scalable, and open solution to one of the most persistent threats in global healthcare. The architecture and implementation decisions documented in this report provide a reproducible foundation for further research and development in this area.

REFERENCES

1. World Health Organization 2017, Falsified Medicines Report, WHO Press, Geneva, Switzerland.
2. MediLedger Project 2020, DSCSA Pilot Report, Chronicled Inc., San Francisco, CA, USA.
3. Li, X, Wang, Y & Zhang, H 2020, 'Pharmaceutical counterfeit detection using convolutional neural networks applied to packaging images', Computers in Biology and Medicine, vol. 118, p. 103623.
4. Wang, Z, Bovik, AC, Sheikh, HR & Simoncelli, EP 2004, 'Image quality assessment: From error visibility to structural similarity', IEEE Transactions on Image Processing, vol. 13, no. 4, pp. 600-612.
5. Rublee, E, Rabaud, V, Konolige, K & Bradski, G 2011, 'ORB: An efficient alternative to SIFT or SURF', Proceedings of the IEEE International Conference on Computer Vision (ICCV), Barcelona, Spain, pp. 2564-2571.
6. Benet, J 2014, 'IPFS — Content addressed, versioned, P2P file system', arXiv preprint arXiv:1407.3561.
7. Szabo, N 1996, 'Smart contracts: Building blocks for digital markets', Extropy: Journal of Transhumanist Thought, vol. 16.
8. Kumar, R & Tripathi, R 2019, 'Traceability of counterfeit medicine supply chain through Blockchain', 2019 11th International Conference on Communication Systems and Networks (COMSNETS), Bengaluru, India, pp. 568-570.
9. Tseng, JH, Liao, YC, Chong, B & Liao, SW 2018, 'Governance on the drug supply chain via Gcoin blockchain', International Journal of Environmental Research and Public Health, vol. 15, no. 6, p. 1055.
10. Abbas, S, Bilal, M, Ahmad, M, Ali Khan, M, Islam, S, Khan, A, Seo, JT & Whangbo, T 2020, 'A blockchain and machine learning-based drug supply chain management and recommendation system for smart pharmaceutical industry', Electronics, vol. 9, no. 5, p. 852.
11. Rai, BK 2022, 'BBTCD: Blockchain based traceability of counterfeited drugs', Journal of The Institution of Engineers (India): Series B, vol. 103, pp. 475-486.

12. Musamih, A, Salah, K, Jayaraman, R, Arshad, J, Debe, M, Al-Hammadi, Y & Ellahham, S 2021, 'A blockchain-based approach for drug traceability in healthcare supply chain', IEEE Access, vol. 9, pp. 97750-97763.
13. Zhu, Q, Bai, L & Zhang, K 2020, 'A blockchain based solution for medication anti-counterfeiting and traceability', IEEE Access, vol. 8, pp. 184256-184268.
14. Singh, A, Dwivedi, P & Srivastava, G 2020, 'IoT based blockchain for temperature monitoring and counterfeit pharmaceutical prevention', SSRN Electronic Journal.
15. Sylim, P, Liu, F, Marcelo, A & Fontelo, P 2018, 'Blockchain technology for detecting falsified and substandard drugs in distribution: Pharmaceutical supply chain intervention', JMIR Research Protocols, vol. 7, no. 9, p. e10163.
16. Uddin, M 2021, 'Blockchain Medledger: Hyperledger fabric enabled drug traceability system for counterfeit drugs in pharmaceutical industry', International Journal of Pharmaceutics, vol. 597, p. 120235.
17. Gomasta, SS, Dhar, T, Paul, N, Whaiduzzaman, M & Uddin, MS 2023, 'PharmaChain: Blockchain-based drug supply chain provenance verification system', Applied Sciences, vol. 13, no. 7, p. 4219.
18. Al-Farsi, A, Bensmail, H & Bakiras, S 2022, 'Securing blockchain-based supply chain workflow against internal and external attacks', IEEE Access, vol. 10, pp. 40525-40541.

APPENDIX A

FINAL YEAR PROJECT REGISTRATION FORM

We hereby affirm the originality and authenticity of the Final Year Project, "MedSecure: A Blockchain and Artificial Intelligence-Powered Framework for Pharmaceutical Supply Chain Integrity" to be undertaken will be upheld. The report and/or system that we submit at the conclusion of the Final Year Project will be the result of our own investigations and efforts.

We understand that cheating and plagiarism constitute a serious violation of the University regulations, which will not only result in a failing grade for the Final Year Project but subject to further disciplinary actions.

_____

Signature of Students

Full Names of Students :	Tanisha Katariya, Ayushi Patel, Krutang Panchal, Vraj Sondagar
Enrolment Numbers :	22C21028, 22C21067, 22C21045, 22C21117
School :	School of Computer Science Engineering & Technology
Programme :	BTech CSE
Specialization :	Computer Science Engineering
Project Domain :	Blockchain and Artificial Intelligence
Project Title :	MedSecure: A Blockchain and AI-Powered Framework for Pharmaceutical Supply Chain Integrity

_____

Signature of Supervisor

Name: _____

Date: _____

MedSecure: A Blockchain and Artificial Intelligence-Powered Framework for Pharmaceutical Supply Chain Integrity and Counterfeit Medicine Detection

Vraj Sondagar, Tanisha Katariya, Ayushi Patel, Krutang Panchal, Prof. Shivangi Matieda

Dept. of Computer Science Engineering, ITM SLS Baroda University, Vadodara, Gujarat, India

Abstract—The global pharmaceutical supply chain faces an escalating crisis of counterfeit medicines, which the World Health Organization estimates account for approximately 10% of medicines in low- and middle-income countries, causing hundreds of thousands of preventable deaths annually. Existing traceability solutions remain fragmented, relying on centralised databases susceptible to data tampering, single points of failure, and limited cross-stakeholder transparency. This paper proposes MedSecure, a decentralised framework integrating Ethereum blockchain technology with an artificial intelligence-driven computer vision module to deliver end-to-end pharmaceutical supply chain integrity and real-time counterfeit detection. Manufacturers register medicine batches immutably on-chain with IPFS-linked documentation, and a smart-contract-enforced state machine governs ownership transfers across manufacturers, distributors, and pharmacies. At the point of sale, consumers and pharmacists verify Batch IDs to trigger real-time on-chain verification supplemented by an OpenCV-based packaging authentication pipeline employing ORB feature matching and the Structural Similarity Index Measure (SSIM). The proposed composite confidence scoring achieves reliable discrimination between authentic and counterfeit packaging. System architecture, core algorithms, security analysis, and comparative evaluation demonstrate that MedSecure offers a scalable, cost-effective, and regulator-compliant solution to pharmaceutical counterfeiting.

Keywords—*Blockchain; counterfeit medicine detection; pharmaceutical supply chain; computer vision; Ethereum smart contracts; IPFS; ORB; SSIM*

I. INTRODUCTION

Counterfeit and substandard medicines represent one of the most serious threats to global public health. The World Health Organization reported that falsified medical products are prevalent in both developed and developing economies, with an estimated market value exceeding USD 200 billion annually. Patients who consume counterfeit medicines receive either no therapeutic benefit or, in many cases, actively harmful substances, contributing to antimicrobial resistance, treatment failure, and death. In low- and middle-income countries, falsified medicines are estimated to cause over 250,000 deaths each year, with a disproportionate impact on children suffering from malaria and pneumonia.

The pharmaceutical supply chain is inherently complex, spanning manufacturers, distributors, wholesalers, pharmacies, and consumers across national and international borders. This complexity creates multiple points of vulnerability at which counterfeit products may be introduced. Traditional regulatory approaches rely on paper-based certificates and centralised digital databases, both of which

are susceptible to forgery and tampering. The absence of interoperability between stakeholder systems means that end-to-end traceability from manufacturer to consumer remains largely unachievable in practice.

Blockchain technology offers a promising architectural foundation for supply chain transparency. Its properties of immutability, decentralisation, and cryptographic verifiability make it well-suited to maintaining tamper-proof records of pharmaceutical product provenance. However, blockchain alone does not address physical packaging authentication—a counterfeit product may carry a legitimate Batch ID if that code has been copied from authentic packaging. Artificial intelligence, particularly computer vision, has demonstrated significant promise in detecting packaging anomalies indicative of counterfeiting. Feature-matching algorithms such as ORB and perceptual similarity metrics such as SSIM have been applied to product authentication tasks with encouraging results.

This paper presents MedSecure, a novel framework integrating Ethereum smart contracts, the InterPlanetary File System (IPFS), and an OpenCV-based computer vision module into a unified pharmaceutical verification platform. The system combines immutable on-chain state management, tamper-evident off-chain storage, keyed Batch ID integrity verification, and AI-powered packaging authentication to address the principal limitations of existing pharmaceutical traceability systems. The remainder of this paper is structured as follows: Section II surveys related work; Section III describes the proposed system architecture; Section IV presents core algorithms; Section V provides security analysis; Section VI discusses comparative evaluation and implementation results; and Section VII concludes the paper.

II. LITERATURE REVIEW

A. Blockchain-Based Pharmaceutical Traceability

The application of blockchain to pharmaceutical supply chains was first systematically explored in the context of the United States Drug Supply Chain Security Act (DSCSA). MediLedger, developed by Chronicled Inc., demonstrated a permissioned blockchain network through which pharmaceutical manufacturers and wholesalers could verify product provenance and perform regulatory recall management. The MediLedger pilot established that blockchain-based traceability was technically viable at commercial scale, though it was limited to inter-enterprise B2B transactions and did not extend verification capabilities to the consumer.

Hyperledger Fabric-based systems have also been proposed for pharmaceutical supply chain management. These permissioned architectures offer lower transaction latency compared to public blockchains but sacrifice the openness and verifiability properties that make public chains particularly resistant to collusion between supply chain participants. The Ethereum-based approach adopted in

MedSecure leverages the transparency of a public blockchain, ensuring audit records are verifiable by any party without reliance on a trusted consortium operator.

B. Artificial Intelligence for Counterfeit Detection

Convolutional neural networks have been applied to medical packaging image classification, achieving accuracy rates above 90% on controlled datasets. While deep learning approaches offer strong classification performance, they require substantial labelled training data and significant computational resources at inference time, limiting deployment in low-resource pharmacy settings. Feature-based approaches such as ORB and SIFT offer a compelling alternative: they are computationally efficient, rotation and scale invariant, and require no prior domain-specific training data.

SSIM was originally proposed as a perceptual image quality metric and has since been widely adopted for image similarity assessment in manufacturing quality control. Its sensitivity to luminance, contrast, and structural differences makes it particularly well-suited to detecting the print quality deviations, colour shifts, and font anomalies characteristic of counterfeit pharmaceutical packaging.

C. Hybrid Blockchain and AI Approaches

A small number of prior works have explored integration of blockchain and AI for supply chain authentication. However, most existing proposals remain conceptual or address narrow subproblems. MedSecure advances this line of work by implementing a fully operational, end-to-end integrated system in which the blockchain serves as the authoritative source of truth, IPFS provides tamper-evident off-chain storage, and the AI module directly queries IPFS-stored reference images during real-time verification. This integration represents a novel contribution to the field, combining previously separate capabilities into a unified, consumer-accessible framework.

III. PROPOSED SYSTEM ARCHITECTURE

MedSecure is designed as a layered, modular architecture comprising four principal layers: the blockchain and smart contract layer, the decentralised storage layer, the application service layer, and the frontend presentation layer. Each layer encapsulates distinct responsibilities and communicates through well-defined interfaces, enabling independent scalability and maintainability.

A. Blockchain and Smart Contract Layer

The Ethereum blockchain serves as the immutable ledger for all critical supply chain events. A Solidity smart contract governs the complete lifecycle of each medicine batch through a strict linear state machine: MANUFACTURED → SENT_TO_DISTRIBUTOR → READY_FOR_SALE → SOLD. State transitions are enforced on-chain, ensuring that no actor can skip a stage or reverse a transition regardless of application-layer behaviour. Role-based access control is implemented within the contract; each wallet address is assigned one of four roles (Manufacturer, Distributor, Pharmacy,

Consumer) and may only invoke functions appropriate to that role. The contract is developed using Solidity 0.8.x and deployed to the Ethereum Sepolia public testnet using the Foundry development framework, which provides gas-optimised compilation and integrated unit testing.

B. Decentralised Storage Layer (IPFS)

All batch documentation—including compliance certificates, packaging specifications, and reference packaging images—is stored on IPFS through the Pinata pinning service. Each uploaded file is assigned a unique content identifier (CID) derived from the cryptographic hash of the file contents. The CID is subsequently recorded on the Ethereum blockchain during batch registration, creating a tamper-evident link between the on-chain record and off-chain documents. Any post-registration modification to a stored file produces a different CID, rendering tampering immediately detectable. This content-addressed approach eliminates the need for centralised file servers and ensures that reference images used during consumer verification remain immutable.

C. Application Service Layer

The application service layer is implemented as a stateless Next.js API service providing RESTful endpoints for all supply chain operations. The service layer interfaces with the Ethereum network through the Viem and Wagmi libraries, which manage wallet connection, transaction signing, and contract interaction. A PostgreSQL database serves as a read-optimised mirror of on-chain data, caching batch metadata and Batch ID records to reduce on-chain query costs and improve response latency for high-frequency read operations. The blockchain remains the authoritative source of truth; PostgreSQL serves exclusively as a performance cache with indexed columns on batchId and walletAddress for fast query performance.

The computer vision authentication module is implemented as a separate Python microservice using OpenCV. This stateless service accepts a base64-encoded live packaging image and a batch identifier, retrieves corresponding reference images from IPFS using the on-chain manifestCID, executes the authentication pipeline, and returns a confidence score and classification result. The microservice is designed to be horizontally scalable, allowing deployment across multiple instances as verification demand increases.

D. Frontend Presentation Layer

Four distinct portal interfaces serve the four system actors. The Manufacturer Portal provides batch registration, document upload, and Batch ID generation workflows with real-time validation feedback. The Distributor Portal presents incoming transfer notifications from manufacturers and enables outbound transfer initiation to pharmacies. The Pharmacy Portal enables batch acceptance, BATCH ID verifying, and sale completion with colour-coded verification status. The Consumer Portal—accessible via any modern browser without a Web3 wallet—provides a simplified BATCH ID verification

interface with integrated computer vision verification and plain-language authenticity results in clear, colour-coded terms (green for VERIFIED, amber for SUSPICIOUS, red for UNSAFE). This wallet-free design is a significant departure from existing enterprise-focused systems and represents a key contribution enabling direct public access to supply chain verification.

IV. CORE ALGORITHMS AND IMPLEMENTATION

A. Batch Registration Algorithm

The batch registration algorithm coordinates three operations: IPFS document upload, on-chain metadata persistence, and Batch ID generation. Input documents and packaging images are each uploaded to IPFS individually, yielding per-file content identifiers. A JSON manifest aggregating all CIDs and batch metadata is then uploaded as a single document, producing a manifestCID that serves as the canonical on-chain reference. The smart contract's registerBatch function is invoked with the manifestCID and key metadata (manufacturing date, expiry date, batch composition), assigning a unique batchId and emitting a BatchRegistered event.

B. Computer Vision Authentication Algorithm

The CV authentication algorithm employs two complementary techniques: ORB feature matching and the Structural Similarity Index Measure (SSIM). Reference packaging images are retrieved from IPFS using the on-chain manifestCID. Both live and reference images are subjected to a standardised pre-processing pipeline comprising greyscale conversion, Gaussian blur, Contrast Limited Adaptive Histogram Equalisation (CLAHE), and Canny edge detection, ensuring robustness to lighting variation at the point of purchase.

ORB is applied to detect up to 500 keypoints per image and compute binary descriptors. A brute-force matcher computes the ratio of good matches (Hamming distance below threshold 50) to total matches, yielding an ORB score for each reference image. In parallel, SSIM measures perceptual similarity across luminance, contrast, and structural dimensions. The final confidence score is the equally weighted average of the maximum ORB score and the maximum SSIM score across all reference images, computed as: confidence = $0.5 \times ORB_{max} + 0.5 \times SSIM_{max}$. Batches scoring 75% or above are classified as AUTHENTIC, those scoring 50%–74% as SUSPICIOUS, and those below 50% as COUNTERFEIT. The result and confidence score are written to both PostgreSQL and the Ethereum smart contract, maintaining an immutable verification audit trail.

C. Batch ID Lifecycle Management

Batch IDs are generated at batch registration with a keyed SHA-256 checksum payload at high error-correction level H. Upon verifying, the checksum is recomputed and compared; a mismatch indicates digital tampering of the BATCH ID payload. Where the checksum is valid, the system checks

whether the BATCH ID record is flagged as locked—indicating a previously completed sale. A locked Batch ID returns an ALREADY_SOLD response, preventing reuse of Batch IDs removed from sold packaging. Upon sale completion via the markAsSold smart contract function, the batch transitions to SOLD status and the BATCH ID record is permanently locked on-chain, providing an effective defence against BATCH ID cloning attacks.

V. SECURITY ANALYSIS

MedSecure addresses several well-known attack vectors in pharmaceutical supply chain systems through a defence-in-depth architecture. Blockchain immutability ensures that batch registration and ownership transfer records cannot be retroactively altered by any single party. Role-based smart contract enforcement eliminates the possibility of unauthorised state transitions at the application layer. IPFS content addressing renders document tampering immediately detectable, as any modification to a stored file produces a different CID inconsistent with the on-chain record.

Batch ID locking prevents double-spend attacks in which a counterfeit product carries a cloned Batch ID from a legitimately sold unit. The SHA-256 keyed checksum detects payload substitution attacks. The dual-layer verification approach—combining Batch ID validation with computer vision packaging authentication—creates a defence-in-depth architecture: a counterfeit product that successfully copies a legitimate Batch ID must also replicate the reference packaging images with sufficient fidelity to pass computer vision checks. The ORB and SSIM algorithms are specifically chosen because they detect minute variations in print quality, colour consistency, and font rendering—characteristics that high-fidelity counterfeiting equipment struggles to replicate.

VI. RESULTS AND COMPARATIVE EVALUATION

System testing confirmed that MedSecure met all specified functional and non-functional requirements. Smart contract unit tests implemented in Foundry achieved full coverage of all state transition paths across the four actor roles and five batch status states, including all expected revert conditions. All test cases passed with successful enforcement of the state machine and role-based access control.

The computer vision authentication module was tested against a controlled dataset of packaging samples comprising authentic reference images, digitally altered counterfeit packaging (with logo, text, and colour modifications), and physically degraded authentic packaging (with partial surface obscuring). The composite ORB and SSIM scoring demonstrated reliable discrimination between authentic and counterfeit samples. Authentic samples consistently scored above 75% confidence. Counterfeit samples with significant packaging alterations scored below 50% confidence in all cases. The CLAHE pre-processing effectively normalised lighting variation across test samples, with authentic images remaining verifiable even under diverse ambient lighting conditions.

The Batch ID integrity verification mechanism successfully detected all payload substitution attacks (modified checksum payloads) and correctly identified all post-sale scans as ALREADY_SOLD. No false negatives were observed. Performance evaluation showed BATCH ID verify to on-chain data retrieval completed in 1.2–1.8 seconds, and the full CV authentication pipeline (IPFS + pre-processing + ORB + SSIM) completed in 3.1–4.7 seconds, well within target thresholds.

Comparative evaluation against existing pharmaceutical traceability and anti-counterfeiting systems confirmed that MedSecure is the only evaluated system providing simultaneous public blockchain deployment, direct consumer verification access without wallet requirement, computer vision-based physical packaging authentication, and Batch ID locking within a single integrated framework. MediLedger, whilst pioneering in consortium blockchain approaches, lacks consumer access and CV authentication. BBTCD provides blockchain and IPFS but lacks consumer verification and CV authentication. Abbas et al. and other ML-based approaches lack public blockchain deployment and consumer access.

VII. CONCLUSION

This paper has presented MedSecure, a blockchain and artificial intelligence-powered framework for pharmaceutical supply chain integrity and counterfeit medicine detection. By integrating Ethereum smart contracts deployed on the Sepolia public testnet, IPFS decentralised storage, an OpenCV-based computer vision authentication module, and a consumer-facing verification portal, MedSecure addresses the principal limitations of existing pharmaceutical traceability and anti-counterfeiting systems.

The system's defence-in-depth architecture—combining immutable on-chain state management through a Solidity-enforced state machine, tamper-evident IPFS document and image storage with on-chain CID anchoring, keyed SHA-256 Batch ID integrity verification with permanent post-sale locking, and AI-powered ORB and SSIM packaging authentication—provides multiple independent layers of assurance that are substantially more difficult for counterfeiters to circumvent simultaneously than any single-layer approach.

Future work will explore deployment on Ethereum Layer-2 networks such as Optimism or Arbitrum to reduce per-transaction gas costs by approximately one order of magnitude, making mainnet deployment economically viable for pharmaceutical manufacturers. Integration of deep learning-based packaging classifiers—such as fine-tuned EfficientNet or Vision Transformers—as supplementary authentication layers would improve detection of sophisticated high-fidelity counterfeits. Integration with national drug regulatory authority databases would enable automated cross-referencing of batch records against approved product lists, further strengthening the regulatory compliance layer.

MedSecure demonstrates that the convergence of public blockchain technology, decentralised storage, and computer vision can provide a practical, scalable, and open solution to one of the most

persistent threats in global healthcare. The architecture and implementation decisions documented in this paper provide a reproducible foundation for further research and development in this area.

REFERENCES

[1] World Health Organization, "Falsified Medicines Report," WHO Press, Geneva, Switzerland, 2017.

[2] MediLedger Project, "DSCSA Pilot Report," Chronicled Inc., San Francisco, CA, USA, 2020.

[3] X. Li, Y. Wang, and H. Zhang, "Pharmaceutical counterfeit detection using convolutional neural networks applied to packaging images," Computers in Biology and Medicine, vol. 118, p. 103623, 2020.

[4] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Transactions on Image Processing, vol. 13, no. 4, pp. 600–612, 2004.

[5] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in Proceedings of the IEEE International Conference on Computer Vision (ICCV), Barcelona, Spain, 2011, pp. 2564–2571.

[6] J. Benet, "IPFS — Content addressed, versioned, P2P file system," arXiv preprint arXiv:1407.3561, 2014.

[7] N. Szabo, "Smart contracts: Building blocks for digital markets," Extropy: Journal of Transhumanist Thought, vol. 16, 1996.

[8] R. Kumar and R. Tripathi, "Traceability of counterfeit medicine supply chain through Blockchain," in Proceedings of the 11th International Conference on Communication Systems and Networks (COMSNETS), Bengaluru, India, 2019, pp. 568–570.

[9] B. K. Rai, "BBTCD: Blockchain based traceability of counterfeited drugs," Journal of The Institution of Engineers (India): Series B, vol. 103, pp. 475–486, 2022.

[10] A. Musamih, K. Salah, R. Jayaraman, J. Arshad, M. Debe, Y. Al-Hammadi, and S. Ellahham, "A blockchain-based approach for drug traceability in healthcare supply chain," IEEE Access, vol. 9, pp. 97750–97763, 2021.